



Performance Analysis

[Reattempt](#)[Download Certificate](#)

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.

Answer Distribution

0

Correct

1

Incorrect

64

Unattempted

Domain Performance

Database	0%
Cost Optimization	0%
Networking	0%
Compute	0%
Storage	0%
Automation	0%
Security	0%
Monitoring and Observability	0%

Final Score

**0/65**

Passing score: 48

Time Taken

**130m 0s**

Total time: 130 minutes

Question Analysis

✓ Correct	0
✗ Incorrect	1
⊖ Unattempted	64
🔖 Reviewed	0

Question-wise Analysis

Expand All

Collapse All

Question 1

Database



A company's ecommerce application runs on Amazon EC2 instances behind an Application Load Balancer, within an Auto Scaling group spanning multiple Availability Zones. The Auto Scaling group scales based on CPU utilization. Transaction data is stored in a MySQL 8.0 database on a large EC2 instance. Database performance degrades with increased application load. The application is read-heavy. The company needs a solution for automatic database scaling to handle unpredictable read workloads while maintaining high availability. Which solution meets these requirements?

- ⊗ Use Amazon Redshift with a single node for leader and compute functionality.

Amazon Redshift is not MySQL-compatible and using a single node does not provide high availability.

- ⊗ Use Amazon RDS with a Single-AZ deployment. Configure Amazon RDS to add reader instances in a different Availability Zone.

A Single-AZ deployment does not provide high availability. Although read replicas can be created in different AZs, the primary instance remains vulnerable.

- ✔ Use Amazon Aurora with a Multi-AZ deployment. Configure Aurora Auto Scaling with Aurora Replicas.

Aurora is MySQL-compatible, offers high performance, and with Multi-AZ and Auto Scaling Replicas, it meets the requirements of high availability and automatic scaling for read-heavy workloads.

- ⊗ Use Amazon ElastiCache for Memcached with EC2 Spot Instances.

ElastiCache is a caching solution, not a database. It does not meet the requirement of storing transaction data. Also, Spot Instances are not ideal for a production database due to potential interruptions.

Overall Explanation:

Amazon Aurora with a Multi-AZ deployment and Aurora Auto Scaling Replicas is the best solution. Aurora is MySQL-compatible, provides high performance, and its Multi-AZ feature ensures high availability. Aurora Auto Scaling automatically adjusts the number of read replicas to handle unpredictable read workloads, providing the necessary scalability and availability for the application.

 View Discussions

 Ask AI

Question 2

Not Attempted

Cost Optimization



A solutions architect is tasked with optimizing the cost of an application running on AWS. The application utilizes Amazon EC2 instances, AWS Fargate, and AWS Lambda for compute services. The EC2 instances handle the data ingestion layer, which experiences sporadic and unpredictable workloads that can be interrupted. The application's front end operates on Fargate, while Lambda serves as the API layer. The utilization patterns for the front end and API

layer are predictable for the next year. Which combination of purchasing options would provide the MOST cost-effective solution for hosting this application? (Choose two.)

- ☒ Use Spot Instances for the data ingestion layer.
Spot Instances offer significant cost savings for workloads that can tolerate interruptions and have sporadic usage patterns, making them ideal for the data ingestion layer.
- ☐ Use On-Demand Instances for the data ingestion layer.
On-Demand Instances are more expensive and less suitable for workloads with sporadic and interruptible nature.
- ☒ Purchase a 1-year Compute Savings Plan for the front end and API layer.
Compute Savings Plans offer cost savings for consistent compute usage across EC2, Fargate, and Lambda, making them suitable for the predictable front-end and API layers.
- ☐ Purchase a 1-year All Upfront Reserved Instances for the data ingestion layer.
Reserved Instances are less ideal for unpredictable workloads like the data ingestion layer, as they require a commitment to a specific instance type and utilization.
- ☐ Purchase a 1-year EC2 instance Savings Plan for the front end and API layer.
EC2 Instance Savings Plans only apply to EC2 usage and do not cover Fargate and Lambda. Compute Savings Plan is more flexible and provides coverage across these services.

Overall Explanation:

The most cost-effective solution involves using Spot Instances for the data ingestion layer due to its tolerance for interruptions and unpredictable usage and purchasing a 1-year Compute Savings Plan for the front end and API layer because of their predictable utilization and the plan's applicability across Fargate and Lambda. EC2 Instance Savings plans does not include Lambda or Fargate.

[View Discussions](#)[Ask AI](#)

Question 3

Not Attempted

Networking



A company operates a global web portal, delivering personalized breaking news, local alerts, and weather updates using a mix of static and dynamic content. This content is served over HTTPS via an API server running on an Amazon EC2 instance behind an Application Load

Balancer (ALB). The company aims to minimize latency for users worldwide. What is the most effective architectural design for achieving this goal?

- ☒ Deploy the application stack in a single AWS Region. Use Amazon CloudFront to serve all static and dynamic content by specifying the ALB as an origin.
- Deploying in a single region simplifies management. CloudFront, as a CDN, caches content at edge locations globally, reducing latency for users. CloudFront can also be used for dynamic content to reduce latency.
- ☐ Deploy the application stack in two AWS Regions. Use an Amazon Route 53 latency routing policy to serve all content from the ALB in the closest Region.
- While Route 53 latency-based routing directs users to the closest region, deploying in only two regions will not provide the lowest latency to all users across the globe. Also, CloudFront offers better latency reduction.
- ☐ Deploy the application stack in a single AWS Region. Use Amazon CloudFront to serve the static content. Serve the dynamic content directly from the ALB.
- Serving dynamic content directly from the ALB without CloudFront's caching and edge location benefits will result in higher latency compared to using CloudFront for both static and dynamic content.
- ☐ Deploy the application stack in two AWS Regions. Use an Amazon Route 53 geolocation routing policy to serve all content from the ALB in the closest Region.
- Similar to option B, deploying in only two regions is insufficient for global low latency. While geolocation routing is useful, CloudFront provides broader coverage and better caching capabilities.

Overall Explanation:

The optimal solution is to deploy the application stack in a single region and utilize Amazon CloudFront for both static and dynamic content. CloudFront's global network of edge locations caches content closer to users, reducing latency. While Route 53 routing policies can distribute traffic, CloudFront's caching mechanism offers superior performance for both static and dynamic content delivery across a global user base.

[View Discussions](#)[Ask AI](#)

Question 4

Not Attempted

Networking



A gaming company is designing a highly available architecture. The application runs on a modified Linux kernel and supports only UDP-based traffic. The company needs the front-end tier to provide the best possible user experience. That tier must have low latency, route traffic to the nearest edge location, and provide static IP addresses for entry into the application endpoints. What should a solutions architect do to meet these requirements?

- ☐ Configure Amazon Route 53 to forward requests to an Application Load Balancer. Use AWS Lambda for the application in AWS Application Auto Scaling.

Amazon Route 53 doesn't support UDP traffic. Lambda might not be suitable for long-running UDP-based applications.

- ☐ Configure Amazon CloudFront to forward requests to a Network Load Balancer. Use AWS Lambda for the application in an AWS Application Auto Scaling group.

Amazon CloudFront primarily designed for HTTP/HTTPS traffic and does not have native support for UDP-based traffic.

- ☒ Configure AWS Global Accelerator to forward requests to a Network Load Balancer. Use Amazon EC2 instances for the application in an EC2 Auto Scaling group.

AWS Global Accelerator provides static IP addresses and routes traffic to the nearest edge location. Network Load Balancer supports UDP traffic. EC2 instances allow customization and support for UDP and modified kernel. EC2 Auto Scaling provides HA.

- ☐ Configure Amazon API Gateway to forward requests to an Application Load Balancer. Use Amazon EC2 instances for the application in an EC2 Auto Scaling group.

Amazon API Gateway is primarily used for RESTful APIs and does not support UDP-based traffic.

Overall Explanation:

AWS Global Accelerator provides static IP addresses that serve as a fixed entry point and routes traffic to the nearest edge location, ensuring low latency. Network Load Balancer (NLB) allows support for UDP traffic. Amazon EC2 instances allow customization to the Linux kernel and support for UDP-based traffic. The EC2 instances can be auto scaled based on demand to provide high availability.

 View Discussions

 Ask AI

Question 5

Not Attempted

Compute



A company aims to migrate its existing on-premises monolithic application to AWS, prioritizing minimal changes to the front-end and back-end code while breaking the application into smaller, independently managed applications. Each application will be managed by a different team. The solution must be highly scalable and minimize operational overhead. Which of the following options best fulfills these requirements?

- ☐ Host the application on AWS Lambda. Integrate the application with Amazon API Gateway.

AWS Lambda is suitable for event-driven, serverless functions, but migrating a monolithic application to Lambda might require significant refactoring and might not be ideal for larger applications due to execution time limits. While it reduces operational overhead, the initial refactoring effort could be substantial.

- ☐ Host the application with AWS Amplify. Connect the application to an Amazon API Gateway API that is integrated with AWS Lambda.

AWS Amplify is a front-end development platform and hosting service. While it can integrate with serverless backends like API Gateway and Lambda, it may not be the most straightforward solution for migrating a monolithic application with minimal code changes. Also would require substantial refactoring of backend services.

- ☐ Host the application on Amazon EC2 instances. Set up an Application Load Balancer with EC2 instances in an Auto Scaling group as targets.

While this provides scalability and control, managing EC2 instances directly introduces significant operational overhead, including patching, scaling, and maintaining the infrastructure. This approach doesn't minimize operational overhead as much as other options.

- ☒ Host the application on Amazon Elastic Container Service (Amazon ECS). Set up an Application Load Balancer with Amazon ECS as the target.

Amazon ECS allows containerizing the monolithic application, enabling a 'lift and shift' approach with minimal code changes. It facilitates breaking the application into smaller microservices managed by different teams. The Application Load Balancer distributes traffic across the containers, and ECS manages scaling, minimizing operational overhead compared to managing EC2 instances directly. This allows for a gradual transition from monolith to smaller applications over time.

Overall Explanation:

The best solution is to host the application on Amazon ECS and use an Application Load Balancer. ECS allows for containerization with minimal code changes, facilitates breaking the application into smaller, independently manageable microservices, and provides built-in scaling and management features, minimizing operational overhead. Options A and B

might require substantial refactoring. Option C increases operational overhead due to direct EC2 management. Therefore, Option D provides the most suitable balance between minimal code changes, scalability, and reduced operational overhead.

[View Discussions](#)[Ask AI](#)

Question 6

Not Attempted

Database



A company recently started using Amazon Aurora as the data store for its global ecommerce application. When large reports are run, developers report that the ecommerce application is performing poorly. After reviewing metrics in Amazon CloudWatch, a solutions architect finds that the ReadIOPS and CPUUtilization metrics are spiking when monthly reports run. What is the MOST cost-effective solution?

☐ Migrate the monthly reporting to Amazon Redshift.

Migrating to Amazon Redshift introduces additional costs and complexity, and it may not be necessary to switch to a separate data warehousing service for this specific issue.

☒ Migrate the monthly reporting to an Aurora Replica.

Migrating the monthly reporting to an Aurora Replica can offload the reporting workload from the primary Aurora instance, reducing the impact on its performance during large reports. Using an Aurora Replica provides scalability and allows the replica to handle the read-intensive reporting queries, improving the overall performance of the ecommerce application. Aurora Replicas utilize the same storage as the primary instance so there is no additional storage cost. Replicas can be created and destroyed easily to match reporting needs. The primary Aurora instance size does not need to be changed, avoiding additional cost. Workload is offloaded from the primary instance, improving its performance. No major software/configuration changes needed compared to options like Redshift.

☐ Migrate the Aurora database to a larger instance class.

Simply increasing the instance class of the Aurora database may not be the most cost-effective solution if the performance issue can be resolved by offloading the reporting workload to an Aurora Replica.

☐ Increase the Provisioned IOPS on the Aurora instance.

Increasing the Provisioned IOPS alone may not address the issue of spikes in CPUUtilization during large reports, as it primarily focuses on storage performance rather than overall database performance.

Overall Explanation:

The most cost-effective solution is to migrate the monthly reporting to an Aurora Replica. An Aurora Replica allows you to offload the reporting workload from the primary Aurora instance, reducing the impact on the application's performance. It provides scalability and handles the read-intensive reporting queries without incurring significant additional costs.

[View Discussions](#)[Ask AI](#)

Question 7

Not Attempted

Cost Optimization



A company hosts a website analytics application on a single Amazon EC2 On-Demand Instance. The analytics software is written in PHP and uses a MySQL database. The analytics software, the web server that provides PHP, and the database server are all hosted on the EC2 instance. The application is showing signs of performance degradation during busy times and is presenting 5xx errors. The company needs to make the application scale seamlessly and cost-effectively. Which solution will meet these requirements MOST cost-effectively?

- ⊗ Migrate the database to an Amazon RDS for MySQL DB instance. Create an AMI of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use an Application Load Balancer to distribute the load to each EC2 instance.

This option provides some scaling but is not as seamless or cost-effective as using Auto Scaling and Spot Instances. It only adds a second instance and doesn't address automatic scaling based on demand.

- ⊗ Migrate the database to an Amazon RDS for MySQL DB instance. Create an AMI of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use Amazon Route 53 weighted routing to distribute the load across the two EC2 instances.

Using Route 53 weighted routing is less dynamic than an Application Load Balancer for distributing load and doesn't provide automatic scaling. Also, it only adds a second instance.

- ⊗ Migrate the database to an Amazon Aurora MySQL DB instance. Create an AWS Lambda function to stop the EC2 instance and change the instance type. Create an Amazon CloudWatch alarm to invoke the Lambda function when CPU utilization surpasses 75%.

This option is not a scalable or cost-effective approach. Using Lambda to change the instance type based on CPU utilization is inefficient compared to Auto Scaling.

- ✓ Migrate the database to an Amazon Aurora MySQL DB instance. Create an AMI of the web application. Apply the AMI to a launch template. Create an Auto Scaling group with the launch template. Configure the launch template to use a Spot Fleet. Attach an Application Load Balancer to the Auto Scaling group.

This is the most cost-effective and scalable solution. Migrating to Aurora provides a scalable database. Auto Scaling with a launch template and Spot Fleet enables automatic scaling based on demand, leveraging cheaper Spot Instances. The Application Load Balancer distributes traffic evenly.

Overall Explanation:

Option D provides the most cost-effective and seamlessly scalable solution. Migrating the database to Amazon Aurora MySQL provides a scalable database. Creating an AMI and using it in an Auto Scaling group configured with a launch template and Spot Fleet allows cheap and efficient scaling of the web tier using Spot Instances. The Application Load Balancer distributes traffic across the Auto Scaling group, ensuring high availability and seamless scaling without manual intervention. Options A and B do not scale seamlessly. Option C is an inefficient way to handle scaling.

 View Discussions

 Ask AI

Question 8

Not Attempted

Cost Optimization



A company hosts a stateless web application on Amazon EC2 On-Demand Instances behind an Application Load Balancer. The application experiences peak usage for 8 hours each business day, moderate usage overnight, and low usage during weekends. To minimize EC2 costs without affecting application availability, which solution should the company implement?

- ✗ Use Spot Instances for the entire workload.

Using Spot Instances for the entire workload might affect application availability due to potential interruptions.

- ✓ Use Reserved Instances for the baseline level of usage. Use Spot Instances for any additional capacity that the application needs.

This option provides a cost-effective solution by utilizing Reserved Instances for predictable baseline usage and Spot Instances for fluctuating peak demands, thus minimizing costs without impacting availability.

- ⊗ Use On-Demand Instances for the baseline level of usage. Use Spot Instances for any additional capacity that the application needs.

Using On-Demand Instances for baseline usage is more expensive than using Reserved Instances for the same purpose.

- ⊗ Use Dedicated Instances for the baseline level of usage. Use On-Demand Instances for any additional capacity that the application needs.

Dedicated Instances are the most expensive option and are not necessary for this scenario.

Overall Explanation:

The most cost-effective solution that also maintains availability involves using Reserved Instances for the predictable baseline workload during off-peak hours and weekends, and using Spot Instances to handle peak demand. Spot Instances are well-suited to handle additional load due to the stateless nature of the application. Reserved instances are cheaper than On-Demand instances, especially if an instance is to be running most of the time. Spot instances are the cheapest, but can be terminated.

 View Discussions

 Ask AI

Question 9

Not Attempted

Storage



A company needs to retain application log files for a critical application for 10 years. The application team regularly accesses logs from the past month for troubleshooting, but logs older than 1 month are rarely accessed. The application generates more than 10 TB of logs per month. Which storage option meets these requirements MOST cost-effectively?

- ⊗ Store the logs in Amazon S3. Use AWS Backup to move logs more than 1 month old to S3 Glacier Deep Archive.

Using AWS Backup to move logs to S3 Glacier Deep Archive can incur additional costs and complexity compared to using S3 Lifecycle policies directly. Also, AWS Backup does not support Glacier Deep Archive.

- ✔ Store the logs in Amazon S3. Use S3 Lifecycle policies to move logs more than 1 month old to S3 Glacier Deep Archive.

Storing logs in S3 and using S3 Lifecycle policies to transition logs older than 1 month to S3 Glacier Deep Archive allows for cost optimization based on data access patterns. Since logs older than 1

month are rarely accessed, moving them to S3 Glacier Deep Archive helps minimize storage costs while still retaining the logs for the required 10-year period.

- ⊗ Store the logs in Amazon CloudWatch Logs. Use AWS Backup to move logs more than 1 month old to S3 Glacier Deep Archive.

This adds unnecessary complexity and costs by involving CloudWatch Logs and AWS Backup when direct management through S3 is sufficient. AWS Backup does not support Glacier Deep Archive.

- ⊗ Store the logs in Amazon CloudWatch Logs. Use Amazon S3 Lifecycle policies to move logs more than 1 month old to S3 Glacier Deep Archive.

Using S3 Lifecycle policies to move logs from CloudWatch Logs to S3 Glacier Deep Archive is not a valid option. CloudWatch Logs and S3 have separate storage mechanisms, and S3 Lifecycle policies cannot be applied directly to CloudWatch Logs.

Overall Explanation:

The most cost-effective solution is to store logs in Amazon S3 and use S3 Lifecycle policies to move logs older than 1 month to S3 Glacier Deep Archive. This leverages S3's storage tiers and lifecycle management to optimize costs based on data access patterns. AWS Backup does not directly support Glacier Deep Archive, and storing directly in CloudWatch adds unnecessary cost.

[View Discussions](#)[Ask AI](#)

Question 10

Not Attempted

Automation



A company's data ingestion workflow uses an Amazon SNS topic to notify an AWS Lambda function about new data deliveries. The Lambda function processes and stores the data. Network connectivity issues occasionally cause ingestion failures, requiring manual job reruns. How can a solutions architect ensure that all notifications are eventually processed?

- ⊗ Configure the Lambda function for deployment across multiple Availability Zones.

Deploying across multiple Availability Zones increases availability but doesn't guarantee processing of failed notifications.

- ⊗ Modify the Lambda function's configuration to increase the CPU and memory allocations for the function.

Increasing CPU and memory allocations might improve performance but doesn't address the issue of failed notifications due to network issues.

- ⊗ Configure the SNS topic's retry strategy to increase both the number of retries and the wait time between retries.

While retries can help, SNS has a limit on retry attempts and duration. Messages can be discarded after the maximum retries are exhausted, failing to ensure that all notifications are eventually processed.

- ✓ Configure an Amazon Simple Queue Service (Amazon SQS) queue as the on-failure destination. Modify the Lambda function to process messages in the queue.

This option ensures that failed notifications are sent to an SQS queue, acting as a dead-letter queue. The Lambda function can then process these messages from the queue, guaranteeing eventual processing even after initial failures.

Overall Explanation:

Configuring an SQS queue as the on-failure destination for the SNS topic and modifying the Lambda function to process messages from this queue is the most reliable way to ensure that all notifications are eventually processed. When the Lambda function fails, the notification is sent to the SQS queue. The Lambda function then processes messages in the queue. This method avoids permanent message loss unlike relying solely on SNS retries.

 View Discussions

 Ask AI

Question 11

Not Attempted

Compute



A company has a service that produces event data. The company wants to use AWS to process the event data as it is received. The data is written in a specific order that must be maintained throughout processing. The company wants to implement a solution that minimizes operational overhead. How should a solutions architect accomplish this?

- ✓ Create an Amazon Simple Queue Service (Amazon SQS) FIFO queue to hold messages. Set up an AWS Lambda function to process messages from the queue.

This is the correct solution. SQS FIFO queues guarantee message order, and Lambda functions provide a serverless compute environment, minimizing operational overhead.

- ⊗ Create an Amazon Simple Notification Service (Amazon SNS) topic to deliver notifications containing payloads to process. Configure an AWS Lambda function as a subscriber.

SNS does not guarantee message delivery order, so this solution will not meet the requirement of maintaining the order of the event data.

- ⊗ Create an Amazon Simple Queue Service (Amazon SQS) standard queue to hold messages. Set up an AWS Lambda function to process messages from the queue independently.

SQS standard queues do not guarantee message order, so this solution will not meet the requirement of maintaining the order of the event data.

- ⊗ Create an Amazon Simple Notification Service (Amazon SNS) topic to deliver notifications containing payloads to process. Configure an Amazon Simple Queue Service (Amazon SQS) queue as a subscriber.

While this solution uses both SNS and SQS, SNS does not guarantee message order, and subscribing an SQS standard queue won't either. This solution doesn't guarantee the data will be processed in the correct order.

Overall Explanation:

The correct solution is to use an SQS FIFO queue with a Lambda function. SQS FIFO queues ensure that messages are processed in the order they are received, which fulfills the requirement of maintaining the data's specific order. Lambda minimizes operational overhead by providing a serverless environment. SNS doesn't guarantee message ordering, and standard SQS queues do not either.

[View Discussions](#)[Ask AI](#)

Question 12

Not Attempted

Security



A company recently migrated to AWS and needs to implement a solution to protect the traffic flowing in and out of its production VPC. Previously, an on-premises inspection server handled traffic flow inspection and filtering. The company aims to replicate these functionalities in the AWS Cloud. Which solution best meets these requirements?

- ☐ Use Amazon GuardDuty for traffic inspection and traffic filtering in the production VPC.
Amazon GuardDuty is a threat detection service, not a traffic inspection or filtering service.
- ☐ Use Traffic Mirroring to mirror traffic from the production VPC for traffic inspection and filtering.
Traffic Mirroring replicates traffic but doesn't perform inspection or filtering itself. It requires a separate inspection system.
- ☒ Use AWS Network Firewall to create the required rules for traffic inspection and traffic filtering for the production VPC.
AWS Network Firewall is a managed firewall service that provides filtering for both inbound and outbound network traffic. It allows for creating rules for traffic inspection and filtering.
- ☐ Use AWS Firewall Manager to create the required rules for traffic inspection and traffic filtering for the production VPC.
AWS Firewall Manager helps centrally configure and manage firewalls but doesn't perform traffic inspection or filtering directly.

Overall Explanation:

AWS Network Firewall is the most suitable solution. It allows you to create rules for traffic inspection and filtering directly within the production VPC, replicating the functionalities of the on-premises inspection server. Traffic Mirroring (Option B) only provides a copy of the traffic and requires a separate inspection system. GuardDuty (Option A) is a threat detection service, and Firewall Manager (Option D) is a management service, neither of which directly performs traffic inspection and filtering.

[View Discussions](#)[Ask AI](#)

Question 13

Not Attempted

[Monitoring and Observability](#)

A company is migrating an application from on-premises servers to Amazon EC2 instances. As part of the migration design requirements, a solutions architect must implement infrastructure metric alarms. The company does not need to take action if CPU utilization increases to more than 50% for a short burst of time. However, if the CPU utilization increases to more than 50% and read IOPS on the disk are high at the same time, the company needs to act as soon as

possible. The solutions architect also must reduce false alarms. What should the solutions architect do to meet these requirements?

- ☒ Create Amazon CloudWatch composite alarms where possible.
Composite alarms determine their states by monitoring the states of other alarms. This allows triggering an alarm only when both CPU utilization and read IOPS are high, reducing false alarms.
- ☐ Create Amazon CloudWatch dashboards to visualize the metrics and react to issues quickly.
Dashboards help in monitoring overall health but do not directly address triggering actions based on combined metrics exceeding thresholds.
- ☐ Create Amazon CloudWatch Synthetics canaries to monitor the application and raise an alarm.
Synthetics canaries monitor application behavior and availability but do not address the specific requirement of monitoring CPU utilization and read IOPS to trigger an alarm.
- ☐ Create single Amazon CloudWatch metric alarms with multiple metric thresholds where possible.
Single metric alarms with multiple thresholds cannot correlate across different metrics (CPU and IOPS) to trigger an alarm only when both conditions are met.

Overall Explanation:

Composite alarms allow defining alarms with multiple metrics and conditions, like high CPU AND high read IOPS in this case. This avoids false positives triggered by a single metric spike. Dashboards are for visualization, and Synthetics tests application availability. Single metric alarms cannot correlate across metrics. Therefore, composite alarms allow acting quickly when both CPU and IOPS are high, per the stated need.

[View Discussions](#)[Ask AI](#)

Question 14

Not Attempted

Networking



A company is migrating its on-premises data center to AWS. The company's compliance policies mandate that only the ap-northeast-3 Region can be used. Company administrators are prohibited from allowing VPCs to connect to the internet. Which of the following solutions, when implemented together, will satisfy these requirements? (Choose two.)

- ✔ Use AWS Control Tower to implement data residency guardrails to deny internet access and deny access to all AWS Regions except ap-northeast-3.

AWS Control Tower's data residency guardrails can restrict access to specific regions and deny internet access, contributing to compliance.

- ✘ Use rules in AWS WAF to prevent internet access. Deny access to all AWS Regions except ap-northeast-3 in the AWS account settings.

AWS WAF is designed to protect web applications from common web exploits and does not control access to AWS Regions.

- ✔ Use AWS Organizations to configure service control policies (SCPs) that prevent VPCs from gaining internet access. Deny access to all AWS Regions except ap-northeast-3.

AWS Organizations SCPs can enforce policies that restrict VPCs from accessing the internet and limit access to specific AWS Regions across all accounts in the organization.

- ✘ Create an outbound rule for the network ACL in each VPC to deny all traffic from 0.0.0.0/0. Create an IAM policy for each user to prevent the use of any AWS Region other than ap-northeast-3.

While network ACLs and IAM policies can restrict internet access and limit regional access for users, they require manual configuration and maintenance across all VPCs and users and are not as centrally managed as SCPs.

- ✘ Use AWS Config to activate managed rules to detect and alert for internet gateways and to detect and alert for new resources deployed outside of ap-northeast-3.

AWS Config provides monitoring and alerting capabilities, but it does not actively prevent VPCs from accessing the internet or block the deployment of resources in other regions. It only detects and alerts.

Overall Explanation:

The correct answers are A and C. AWS Control Tower can be used with data residency guardrails to enforce regional restrictions and deny internet access. AWS Organizations with SCPs provides a centralized and effective way to restrict VPCs from gaining internet access and to deny access to all AWS Regions except ap-northeast-3. Control Tower and SCPs together deliver both governance and enforcement to ensure compliance.

 View Discussions

 Ask AI

Question 15

Not Attempted

Database



A company provides training to new employees using a three-tier web application. The application is only accessed for 12 hours each day. The company utilizes an Amazon RDS for MySQL DB instance to store application data and aims to minimize costs. What is the MOST cost-effective solution a solutions architect can implement to meet these requirements?

- ⊗ Configure an IAM policy for AWS Systems Manager Session Manager. Create an IAM role for the policy. Update the trust relationship of the role. Set up automatic start and stop for the DB instance.

This solution uses Session Manager, which is designed for interactive shell access and doesn't directly address the start/stop requirement efficiently. State Manager would be more appropriate, but Lambda/EventBridge is more cost-effective.

- ⊗ Create an Amazon ElastiCache for Redis cache cluster to allow users to access data when the DB instance is stopped. Invalidate the cache after the DB instance is started.

Using ElastiCache is not a direct solution for minimizing RDS costs. It introduces complexity and doesn't address the core requirement of stopping the DB instance when not in use.

- ⊗ Launch an Amazon EC2 instance. Create an IAM role that grants access to Amazon RDS. Attach the role to the EC2 instance. Configure a cron job to start and stop the EC2 instance on the desired schedule.

This solution introduces unnecessary complexity by involving an EC2 instance and cron jobs. Lambda and EventBridge provide a simpler and more cost-effective approach for scheduling tasks.

- ✔ Create AWS Lambda functions to start and stop the DB instance. Create Amazon EventBridge (Amazon CloudWatch Events) scheduled rules to invoke the Lambda functions. Configure the Lambda functions as event targets for the rules.

This is the most cost-effective solution. Lambda functions triggered by EventBridge can automate the start/stop actions based on the 12-hour access period, minimizing costs by only running the DB instance when needed.

Overall Explanation:

The most cost-effective solution is to use AWS Lambda functions triggered by Amazon EventBridge (Amazon CloudWatch Events) scheduled rules. This allows the RDS instance to be automatically started and stopped based on the application's usage schedule, ensuring minimal costs. Option A uses Systems Manager Session Manager which is not for scheduling. Option B introduces ElastiCache without addressing the core problem. Option

C is more complex with EC2 instance and cron jobs and thus more costly than lambda and EventBridge.

[View Discussions](#)[Ask AI](#)

Question 16

Not Attempted

Storage



A company sells ringtones created from clips of popular songs. The files containing the ringtones are stored in Amazon S3 Standard and are at least 128 KB in size. The company has millions of files, but downloads are infrequent for ringtones older than 90 days. The company needs to save money on storage while keeping the most accessed files readily available for its users. Which action should the company take to meet these requirements MOST cost-effectively?

- ☐ Configure S3 Standard-Infrequent Access (S3 Standard-IA) storage for the initial storage tier of the objects.

S3 Standard-IA is designed for infrequently accessed data, but directly storing all files in S3 Standard-IA will impact performance for newer, frequently accessed ringtones and may incur retrieval costs if accessed within 30 days.

- ☐ Move the files to S3 Intelligent-Tiering and configure it to move objects to a less expensive storage tier after 90 days.

S3 Intelligent-Tiering is suitable for unknown access patterns. However, in this case, the access pattern is predictable (infrequent access after 90 days). Intelligent-Tiering also has monitoring and automation costs. While files are larger than 128kb, Lifecycle policies may be more cost-effective.

- ☐ Configure S3 inventory to manage objects and move them to S3 Standard-Infrequent Access (S3 Standard-1A) after 90 days.

S3 Inventory provides a list of objects and their metadata, but does not directly transition objects between storage classes. It would require a separate process (like a script or application) to perform the actual transition, adding complexity.

- ☒ Implement an S3 Lifecycle policy that moves the objects from S3 Standard to S3 Standard-Infrequent Access (S3 Standard-1A) after 90 days.

This option is the most cost-effective because S3 Lifecycle policies automatically transition objects from S3 Standard to S3 Standard-IA after 90 days, aligning with the infrequent access pattern. S3 Standard provides high performance for frequently accessed newer files, and S3 Standard-IA offers lower storage costs for infrequently accessed files without the added costs of Intelligent-Tiering.

Overall Explanation:

S3 Lifecycle policies provide an automated and cost-effective way to manage the storage class transitions based on object age. Since the access pattern is known (infrequent access after 90 days), using a lifecycle policy to move objects from S3 Standard to S3 Standard-IA after 90 days is the optimal approach. S3 Intelligent Tiering is more suited for unknown or changing access patterns. S3 inventory only provides object metadata and does not automate storage transitions. Configuring S3 Standard-IA from the start does not address the need for readily available access for newer, frequently downloaded ringtones.

[View Discussions](#)[Ask AI](#)

Question 17

Not Attempted

Storage



A company needs to save the results from a medical trial to an Amazon S3 repository. The repository must allow a few scientists to add new files and must restrict all other users to read-only access. No users can have the ability to modify or delete any files in the repository. The company must keep every file in the repository for a minimum of 1 year after its creation date. Which solution will meet these requirements?

- ☐ Use S3 Object Lock in governance mode with a legal hold of 1 year.
Governance mode allows users with special permissions to remove the legal hold, potentially allowing modifications or deletions, which does not meet the requirement that 'no users can have the ability to modify or delete any files'.
- ☒ Use S3 Object Lock in compliance mode with a retention period of 365 days.
Compliance mode ensures that no user, including the root user, can delete or modify the files during the retention period. A retention period of 365 days ensures the files are kept for at least one year. While the solution doesn't explicitly address the 'few scientists' requirement, this is the closest option that fulfills the immutability constraint.
- ☐ Use an IAM role to restrict all users from deleting or changing objects in the S3 bucket. Use an S3 bucket policy to only allow the IAM role.
While IAM roles and bucket policies can restrict deletion and modification, they don't enforce a minimum retention period or provide the same level of immutability as S3 Object Lock.
- ☐ Configure the S3 bucket to invoke an AWS Lambda function every time an object is added. Configure the function to track the hash of the saved object so that modified

objects can be marked accordingly.

This option only provides a mechanism for detecting changes but does not prevent them or enforce a retention period. It also adds unnecessary complexity.

Overall Explanation:

S3 Object Lock in compliance mode, with a retention period of 365 days, is the best solution. Compliance mode ensures that no users can delete or modify the objects, meeting the key requirement. The retention period ensures the data is kept for at least one year. While it doesn't explicitly address the 'few scientists' requirement for adding files (IAM policies alongside object lock would be best for a complete solution), this option fulfills the critical immutability constraint outlined in the question.

[View Discussions](#)[Ask AI](#)

Question 18

Not Attempted

Networking



A large media company hosts a web application on AWS. The company wants to start caching confidential media files so that users around the world will have reliable access to the files. The content is stored in Amazon S3 buckets. The company must deliver the content quickly, regardless of where the requests originate geographically. Which solution will meet these requirements?

- ☐ Use AWS DataSync to connect the S3 buckets to the web application.
AWS DataSync is a data transfer service, not a CDN or caching solution.
- ☐ Deploy AWS Global Accelerator to connect the S3 buckets to the web application.
AWS Global Accelerator improves availability and performance for TCP/UDP traffic but lacks content caching features for media delivery.
- ☒ Deploy Amazon CloudFront to connect the S3 buckets to CloudFront edge servers.
Amazon CloudFront is a CDN that caches content at edge locations globally, providing fast and reliable access to users worldwide by serving content from the nearest edge location. It integrates tightly with S3 for secure, durable storage.

- ☐ Use Amazon Simple Queue Service (Amazon SQS) to connect the S3 buckets to the web application.

Amazon SQS is a message queuing service, not designed for content delivery or caching.

Overall Explanation:

Amazon CloudFront is a content delivery network (CDN) service provided by AWS. It caches content at edge locations worldwide, allowing users to access content quickly regardless of their geographic location. By connecting the S3 buckets to CloudFront, the media files can be cached at edge locations, ensuring reliable and fast delivery to users. DataSync is a data transfer service. Global Accelerator improves performance and availability but isn't designed for caching media files. SQS is a message queue service, not for caching or content delivery. Therefore, the correct solution is deploying CloudFront to connect the S3 buckets to CloudFront edge servers.

 View Discussions

 Ask AI

Question 19

Not Attempted

Database



A company ingests batch data from various databases and live streaming data from network sensors and application APIs. They want to consolidate all this data into a central location for business analytics. The requirement is to process the incoming data and stage it in different Amazon S3 buckets. Data analysts will then run ad-hoc queries and import the data into a business intelligence tool to visualize Key Performance Indicators (KPIs). Which combination of services and steps will fulfill these requirements with the LEAST operational overhead? (Choose two.)

- ☒ Use Amazon Athena for one-time queries. Use Amazon QuickSight to create dashboards for KPIs.
- Athena allows running SQL queries directly on data in S3 without needing to manage infrastructure. QuickSight provides a serverless BI tool for easy dashboard creation and KPI visualization.

- ☐ Use Amazon Kinesis Data Analytics for one-time queries. Use Amazon QuickSight to create dashboards for KPIs.
- Kinesis Data Analytics is more suited for continuous stream processing, not ad-hoc querying. QuickSight is a viable option for dashboarding.

- ⊗ Create custom AWS Lambda functions to move the individual records from the databases to an Amazon Redshift cluster.

This option requires significant custom coding and management overhead, as Lambda functions would need to be written and maintained for each data source. Redshift also adds operational complexity.

- ⊗ Use an AWS Glue extract, transform, and load (ETL) job to convert the data into JSON format. Load the data into multiple Amazon OpenSearch Service (Amazon Elasticsearch Service) clusters.

This option involves significant overhead. Converting everything to JSON and using OpenSearch for this particular use case (ad-hoc queries and BI) is not the most efficient or cost-effective approach. It also requires managing OpenSearch clusters.

- ✓ Use blueprints in AWS Lake Formation to identify the data that can be ingested into a data lake. Use AWS Glue to crawl the source, extract the data, and load the data into Amazon S3 in Apache Parquet format.

Lake Formation simplifies data lake creation and management. AWS Glue automates the process of crawling data sources, inferring schemas, and loading data into S3 in an optimized format (Parquet) which is efficient for querying.

Overall Explanation:

The best options are A and E. AWS Lake Formation combined with AWS Glue provides a streamlined and managed approach to building a data lake in S3 with minimal operational overhead. AWS Glue Crawlers identify the data that can be ingested into the data lake. Athena then enables ad-hoc queries on the data stored in S3, and QuickSight provides a means to create dashboards based on that queried data. These services allow business teams to create dashboards without the need to manage servers or provision resources. Options C and D add operational complexity and should be avoided in this case.

 View Discussions

 Ask AI

Question 20

Not Attempted

Database



A company stores data in an Amazon Aurora PostgreSQL DB cluster. The company must store all the data for 5 years and must delete all the data after 5 years. The company also must indefinitely keep audit logs of actions that are performed within the database. Currently, the

company has automated backups configured for Aurora. Which combination of steps should a solutions architect take to meet these requirements?

☐ Take a manual snapshot of the DB cluster.

Taking manual snapshots alone is not a scalable or automated solution for long-term retention and deletion. AWS Backup is a better managed solution.

☐ Create a lifecycle policy for the automated backups.

Lifecycle policies are best managed through AWS Backup, not directly on automated backups.

☐ Configure automated backup retention for 5 years.

Automated backups have a limited retention period (max 35 days), insufficient for the 5-year requirement.

☒ Configure an Amazon CloudWatch Logs export for the DB cluster.

This ensures that audit logs are captured and stored indefinitely in CloudWatch, meeting the requirement for indefinite audit log retention.

☒ Use AWS Backup to take the backups and to keep the backups for 5 years.

AWS Backup allows for centralized management of backups, including setting retention policies for the required 5 years, satisfying the data storage and deletion requirements.

Overall Explanation:

The most suitable solution is to use AWS Backup to manage the Aurora backups with a retention policy of 5 years, and to configure CloudWatch Logs export for indefinite storage of audit logs. AWS Backup provides a fully managed, policy-based backup solution with snapshot scheduling and retention management. CloudWatch Logs export provides a durable and scalable way to store audit logs.

 View Discussions

 Ask AI

Question 21

Not Attempted

Networking



A solutions architect is optimizing a website for an upcoming global musical event. Videos of the performances will be streamed in real time and then will be available on demand to a global online audience. Which AWS service will improve the performance of both the real-time and on-demand streaming?

✓ Amazon CloudFront

Amazon CloudFront is a content delivery network (CDN) that caches content at edge locations worldwide, providing low-latency delivery to a global audience for both real-time and on-demand streaming.

✗ AWS Global Accelerator

AWS Global Accelerator is more suitable for non-HTTP use cases or when static IP addresses are required. While it can improve TCP traffic routing, it's not primarily designed for caching and delivering streaming content like a CDN.

✗ Amazon Route 53

Amazon Route 53 is a DNS service and not designed specifically for streaming video or caching content.

✗ Amazon S3 Transfer Acceleration

Amazon S3 Transfer Acceleration improves upload speeds to Amazon S3 but does not directly enhance streaming performance or deliver content to users globally.

Overall Explanation:

Amazon CloudFront is the correct choice because it's a CDN that caches content at edge locations around the world, bringing the video content closer to viewers, reducing latency, and improving performance for both live streaming and on-demand playback for the global audience. Global Accelerator is better suited for non-HTTP applications or static IP needs. Route 53 handles DNS resolution but doesn't cache content. S3 Transfer Acceleration optimizes uploads to S3, not content delivery.

 View Discussions

 Ask AI

Question 22

Not Attempted

Security



A company is running a publicly accessible serverless application using Amazon API Gateway and AWS Lambda. The application's traffic has recently spiked due to fraudulent requests from botnets. Which TWO steps should a solutions architect take to block requests from unauthorized users?

✓ Create a usage plan with an API key that is shared with genuine users only.

Using an API key in a usage plan can restrict access to the API to only users possessing the key, which can limit fraudulent access, especially from unsophisticated bots. This adds a layer of authentication and authorization.

- ⊗ Integrate logic within the Lambda function to ignore the requests from fraudulent IP addresses.

While possible, implementing this logic in Lambda is difficult due to the dynamic nature of botnets and constantly changing IP addresses. It is not a scalable or effective solution.

- ✔ Implement an AWS WAF rule to target malicious requests and trigger actions to filter them out.

AWS WAF is designed to protect web applications from malicious attacks like botnets. It provides bot detection capabilities and allows you to define rules to block or filter malicious requests based on patterns, IP reputation, and other criteria.

- ⊗ Convert the existing public API to a private API. Update the DNS records to redirect users to the new API endpoint.

Converting to a private API would make the API inaccessible to the general public, including legitimate users. A private API is only accessible within a VPC. This would require significant architectural changes and is not suitable for a publicly accessible application.

- ⊗ Create an IAM role for each user attempting to access the API. A user will assume the role when making the API call.

Creating an IAM role for each user is not feasible for a public API due to the complexity and management overhead. IAM roles are better suited for internal services or applications with a limited number of users.

Overall Explanation:

The best approach involves a combination of proactive security measures. Creating a usage plan with an API key restricts access to only authorized users. Implementing an AWS WAF rule enables the identification and filtering of malicious bot traffic based on various criteria. Options B, D, and E are either impractical or not suitable for a publicly accessible API.

 View Discussions

 Ask AI

Question 23

Not Attempted

Database



A company hosts a data lake on AWS. The data lake consists of data in Amazon S3 and Amazon RDS for PostgreSQL. The company needs a reporting solution that provides data visualization and includes all the data sources within the data lake. Only the company's management team should have full access to all the visualizations. The rest of the company should have only limited access. Which solution will meet these requirements?

- ⊗ Create an analysis in Amazon QuickSight. Connect all the data sources and create new datasets. Publish dashboards to visualize the data. Share the dashboards with the appropriate IAM roles.

Sharing dashboards with IAM roles is not the correct approach within QuickSight. QuickSight uses users and groups for sharing.

- ✔ Create an analysis in Amazon QuickSight. Connect all the data sources and create new datasets. Publish dashboards to visualize the data. Share the dashboards with the appropriate users and groups.

This is the correct approach. QuickSight allows connecting to S3 and RDS, creating visualizations, and sharing them with specific users and groups to control access.

- ⊗ Create an AWS Glue table and crawler for the data in Amazon S3. Create an AWS Glue extract, transform, and load (ETL) job to produce reports. Publish the reports to Amazon S3. Use S3 bucket policies to limit access to the reports.

While Glue can process data, it doesn't directly provide data visualization capabilities. Relying solely on S3 bucket policies for access control might be cumbersome for managing user-specific permissions.

- ⊗ Create an AWS Glue table and crawler for the data in Amazon S3. Use Amazon Athena Federated Query to access data within Amazon RDS for PostgreSQL. Generate reports by using Amazon Athena. Publish the reports to Amazon S3. Use S3 bucket policies to limit access to the reports.

Athena Federated Query can access data in RDS, but Athena is more suitable for querying and not for creating interactive dashboards and managing user-specific access as effectively as QuickSight.

Overall Explanation:

Amazon QuickSight is the most suitable service for creating data visualizations and dashboards from multiple data sources like S3 and RDS. QuickSight's sharing mechanism, based on users and groups, provides fine-grained access control, allowing the management team to have full access while the rest of the company has limited access.

Other options lack the direct visualization capabilities or user-based access controls needed for this scenario.

[View Discussions](#)[Ask AI](#)

Question 24

Not Attempted

Cost Optimization



An e-commerce company's analytics application, hosted on AWS, generates approximately 300 MB of JSON data monthly. They require a disaster recovery (DR) solution for this data, ensuring millisecond accessibility and a 30-day retention period. Which solution is the MOST cost-effective?

☐ Amazon OpenSearch Service (Amazon Elasticsearch Service)

OpenSearch Service is designed for search and analytics and is generally not the most cost-effective solution for DR data storage, especially given the small data volume and accessibility requirement.

☐ Amazon S3 Glacier

While S3 Glacier is cost-effective for archival, standard Glacier retrieval times are too slow. Glacier Instant Retrieval has a minimum 90-day storage duration which negates cost savings for a 30-day retention.

☒ Amazon S3 Standard

S3 Standard provides the required millisecond access and allows for a 30-day retention period. Given the small data size, its cost is minimal and provides the best balance of cost and performance.

☐ Amazon RDS for PostgreSQL

RDS for PostgreSQL is a relational database service and is an overkill for simply backing up JSON data for DR purposes. It would be significantly more expensive than S3 Standard.

Overall Explanation:

S3 Standard is the most cost-effective option for storing the 300MB of JSON data for 30 days with millisecond retrieval requirements. OpenSearch and RDS are overly complex and expensive, while Glacier's retrieval times are too slow or the retention policy would render it more expensive.

[View Discussions](#)[Ask AI](#)

Question 25

Not Attempted

Database



A company has a small Python application that processes JSON documents and outputs the results to an on-premises SQL database. The application runs thousands of times each day. The company wants to migrate the application to the AWS Cloud. The company needs a highly available solution that maximizes scalability and minimizes operational overhead. Which solution will meet these requirements?

- ☐ Place the JSON documents in an Amazon S3 bucket. Run the Python code on multiple Amazon EC2 instances to process the documents. Store the results in an Amazon Aurora DB cluster.

This option requires manual management and scaling of EC2 instances, resulting in higher operational overhead and complexity.

- ☒ Place the JSON documents in an Amazon S3 bucket. Create an AWS Lambda function that runs the Python code to process the documents as they arrive in the S3 bucket. Store the results in an Amazon Aurora DB cluster.

By placing the JSON documents in an S3 bucket, the documents will be stored in a highly durable and scalable object storage service. Using Lambda eliminates the need to manage and provision servers, ensuring scalability and minimizing operational overhead. Storing the results in an Aurora DB cluster ensures high availability and scalability for the processed data.

- ☐ Place the JSON documents in an Amazon Elastic Block Store (Amazon EBS) volume. Use the EBS Multi-Attach feature to attach the volume to multiple Amazon EC2 instances. Run the Python code on the EC2 instances to process the documents. Store the results on an Amazon RDS DB instance.

This approach still involves manual management and scaling of EC2 instances, increasing operational complexity and overhead.

- ☐ Place the JSON documents in an Amazon Simple Queue Service (Amazon SQS) queue as messages. Deploy the Python code as a container on an Amazon Elastic Container Service (Amazon ECS) cluster that is configured with the Amazon EC2 launch type. Use the container to process the SQS messages. Store the results on an Amazon RDS DB instance.

This solution requires managing and scaling an ECS cluster, adding operational overhead and complexity. Utilizing SQS adds complexity to the system.

Overall Explanation:

The optimal solution is to place the JSON documents in an Amazon S3 bucket, trigger an AWS Lambda function upon arrival to process the documents, and store the results in an Amazon Aurora DB cluster. This approach leverages a serverless architecture, maximizing scalability and minimizing operational overhead by eliminating the need to manage EC2 instances or ECS clusters. S3 provides durable storage, Lambda handles processing without server management, and Aurora offers a highly available and scalable database solution.

[View Discussions](#)[Ask AI](#)

Question 26

Not Attempted

Storage



A company is migrating its financial risk modeling workloads to AWS. These workloads rely on High-Performance Computing (HPC) infrastructure and run on Linux. Each HPC workflow uses hundreds of Amazon EC2 Spot Instances and generates thousands of output files, which need to be stored in persistent storage for future analytics and long-term archival. The company needs a high-performance file system integrated with persistent storage that allows all EC2 instances to read and write the datasets and output files, and also facilitates copying on-premises data to the persistent storage. Which combination of AWS services best meets these requirements?

☒ Amazon FSx for Lustre integrated with Amazon S3

Amazon FSx for Lustre is a high-performance, scalable file system optimized for compute-intensive workloads like HPC. It integrates natively with Amazon S3, enabling data to be copied from on-premises to S3 for persistent storage and accessed by EC2 instances for high-speed processing. It also supports Linux-based workloads.

☐ Amazon FSx for Windows File Server integrated with Amazon S3

Amazon FSx for Windows File Server is not suitable because the workloads run on Linux, and this option is designed for Windows file sharing.

☐ Amazon S3 Glacier integrated with Amazon Elastic Block Store (Amazon EBS)

Amazon S3 Glacier is a low-cost archival storage service and is not optimized for the high-performance file system requirements of HPC workloads.

☐ Amazon S3 bucket with a VPC endpoint integrated with an Amazon Elastic Block Store (Amazon EBS) General Purpose SSD (gp2) volume

Using an S3 bucket with a VPC endpoint integrated with an Amazon EBS General Purpose SSD (gp2) volume does not provide the required high-performance file system capabilities for HPC workloads. EBS is block storage and not a distributed filesystem like Lustre. S3 alone does not provide the needed file system interface.

Overall Explanation:

Amazon FSx for Lustre provides a high-performance, scalable file system optimized for compute-intensive workloads like HPC, and is natively integrated with Amazon S3 for persistent storage. This enables the company to copy on-premises data to S3, making it accessible for processing by EC2 instances in a Linux environment. Other options are not suitable because they do not offer the high-performance file system capabilities required for HPC workloads or are not compatible with Linux.

[View Discussions](#)[Ask AI](#)

Question 27

Not Attempted

[Compute](#)

A company is migrating a containerized application from its on-premises environment to AWS. The application is expected to have a large user base shortly after deployment, requiring high scalability and availability. The company seeks a solution that minimizes operational overhead for managing the container deployment. Which solution best meets these requirements?

- ☒ Store container images in an Amazon Elastic Container Registry (Amazon ECR) repository. Use an Amazon Elastic Container Service (Amazon ECS) cluster with the AWS Fargate launch type to run the containers. Use target tracking to scale automatically based on demand.

This solution utilizes ECR for secure container image storage and ECS with Fargate for serverless container execution, minimizing operational overhead. Target tracking enables automatic scaling based on demand.

- ☐ Store container images in an Amazon Elastic Container Registry (Amazon ECR) repository. Use an Amazon Elastic Container Service (Amazon ECS) cluster with the Amazon EC2 launch type to run the containers. Use target tracking to scale automatically based on demand.

While this solution uses ECR for storage and target tracking for scaling, it requires managing EC2 instances, increasing operational overhead.

- ⊗ Store container images in a repository that runs on an Amazon EC2 instance. Run the containers on EC2 instances that are spread across multiple Availability Zones. Monitor the average CPU utilization in Amazon CloudWatch. Launch new EC2 instances as needed.

This option involves managing both the container repository and the EC2 instances, leading to significant operational overhead and complexity.

- ⊗ Create an Amazon EC2 Amazon Machine Image (AMI) that contains the container image. Launch EC2 instances in an Auto Scaling group across multiple Availability Zones. Use an Amazon CloudWatch alarm to scale out EC2 instances when the average CPU utilization threshold is breached.

This solution focuses on managing EC2 instances and scaling based on CloudWatch alarms, which is less efficient and scalable than using Fargate with ECS for containerized applications.

Overall Explanation:

The best solution is to use Amazon ECR for container image storage and Amazon ECS with the Fargate launch type for running the containers. Fargate provides a serverless environment, minimizing operational overhead. Target tracking allows for automatic scaling based on demand, ensuring high availability and scalability. Storing container images in a repository that runs on an Amazon EC2 instance increases operational overhead, because you are responsible for managing the EC2 instance as well as the repository service running on it. Creating an EC2 Amazon Machine Image(AMI) that contains the container image and launching EC2 instances in an auto-scaling group across multiple Availability Zones is a valid method to provision a highly available solution, but it is more appropriate when the workload can be encapsulated inside the AMI.

[View Discussions](#)[Ask AI](#)

Question 28

Not Attempted

[Compute](#)

A company has two applications: a sender application that sends messages with payloads to be processed, and a processing application intended to receive the messages with payloads. The sender application sends approximately 1,000 messages each hour. Message processing can take up to 2 days. If message processing fails, the messages must be retained without impacting the processing of other messages. Which solution meets these requirements while being the MOST operationally efficient?

- ⊗ Set up an Amazon EC2 instance running a Redis database. Configure both applications to use the instance to store, process, and delete messages.

This option requires managing and maintaining an EC2 instance and Redis database, adding operational overhead.

- ⊗ Use an Amazon Kinesis data stream to receive messages from the sender application. Integrate the processing application with the Kinesis Client Library (KCL).

This option introduces complexity with Kinesis data streams and KCL integration, making it less operationally efficient for this use case.

- ✓ Integrate the sender and processor applications with an Amazon Simple Queue Service (Amazon SQS) queue. Configure a dead-letter queue to collect messages that fail to process.

This option leverages SQS for reliable messaging and a dead-letter queue for handling failed messages, offering operational efficiency and meeting the requirements.

- ⊗ Subscribe the processing application to an Amazon Simple Notification Service (Amazon SNS) topic to receive notifications to process. Integrate the sender application to write to the SNS topic.

This option uses SNS, which is better suited for pub/sub scenarios and less ideal for guaranteed message processing and failure handling.

Overall Explanation:

Using Amazon SQS with a dead-letter queue provides a managed, reliable, and operationally efficient solution for message queuing, handling processing failures, and retaining messages. SQS supports at-least-once delivery, ensuring messages are not lost. The DLQ allows retaining messages that repeatedly fail to process, providing visibility into failures for troubleshooting. While the visibility timeout of 12 hours may be a concern, SQS remains the most suitable service among the available options. Other options like Redis on EC2 or Kinesis involve more operational overhead. SNS is less ideal for guaranteed message processing between two applications.

[View Discussions](#)[Ask AI](#)

Question 29

Not Attempted

Security



A solutions architect must design a solution that uses Amazon CloudFront with an Amazon S3 origin to store a static website. The company's security policy requires that all website traffic be inspected by AWS WAF. How should the solutions architect comply with these requirements?

- ☐ Configure an S3 bucket policy to accept requests coming from the AWS WAF Amazon Resource Name (ARN) only.

Configuring an S3 bucket policy to accept requests from the AWS WAF ARN only would bypass the inspection of traffic by AWS WAF. It does not ensure that all website traffic is inspected.

- ☐ Configure Amazon CloudFront to forward all incoming requests to AWS WAF before requesting content from the S3 origin.

AWS WAF is not a destination to forward requests. AWS WAF is directly integrated with CloudFront as a layer to inspect incoming requests.

- ☐ Configure a security group that allows Amazon CloudFront IP addresses to access Amazon S3 only. Associate AWS WAF to CloudFront.

This approach focuses on controlling access to S3 using a security group but doesn't guarantee all traffic is inspected by AWS WAF before accessing S3 content.

- ☒ Configure Amazon CloudFront and Amazon S3 to use an origin access identity (OAI) to restrict access to the S3 bucket. Enable AWS WAF on the distribution.

This is the correct answer. OAI restricts direct access to the S3 bucket, ensuring requests go through CloudFront. Enabling AWS WAF on the CloudFront distribution ensures all incoming traffic is inspected.

Overall Explanation:

The best solution is to configure CloudFront and S3 to use an OAI to restrict direct access to the S3 bucket, ensuring that only CloudFront can access the content. Simultaneously, enable AWS WAF on the CloudFront distribution. This setup ensures that all incoming traffic is inspected for security threats as required by the company's policy and also protect the S3 content by only allowing requests from the OAI.

 View Discussions

 Ask AI

Question 30

Not Attempted

Networking



Organizers for a global event want to publish daily reports as static HTML pages online. These pages are expected to receive millions of views from users worldwide. The files are currently stored in an Amazon S3 bucket. A solutions architect needs to design a solution that is both efficient and effective in delivering these reports. Which of the following actions should the solutions architect take to achieve this goal?

☐ Generate presigned URLs for the files.

Generating presigned URLs allows temporary access to the files, but it doesn't address the scalability and performance requirements needed to serve millions of views globally.

☐ Use cross-Region replication to all Regions.

Cross-Region replication copies the data to multiple regions, but it doesn't inherently improve performance or reduce latency for users accessing the data. It's more for disaster recovery than performance optimization in this scenario.

☐ Use the geoproximity feature of Amazon Route 53.

Amazon Route 53's geoproximity feature routes DNS traffic based on geographic location, but it doesn't provide caching or content delivery capabilities, which are crucial for serving static content to a global audience efficiently.

☒ Use Amazon CloudFront with the S3 bucket as its origin.

Amazon CloudFront, a content delivery network (CDN), caches content at edge locations around the world, bringing the content closer to users and resulting in faster performance. By using the S3 bucket as the origin, CloudFront can fetch the files once and then serve them from its edge locations.

Overall Explanation:

The most efficient and effective solution is to use Amazon CloudFront with the S3 bucket as its origin. CloudFront is a content delivery network (CDN) that caches content at edge locations globally. This ensures faster delivery and a better user experience for users accessing the static HTML pages from around the world. It also effectively manages the high traffic and large number of requests expected for the global event.

 View Discussions

 Ask AI

A company runs a production application on a fleet of Amazon EC2 instances. The application reads data from an Amazon SQS queue and processes the messages in parallel. The message volume is unpredictable and experiences intermittent traffic. The application must continually process messages without any downtime. Which solution meets these requirements MOST cost-effectively?

- ☐ Use Spot Instances exclusively to handle the maximum capacity required.

Using Spot Instances exclusively may lead to downtime because Spot Instances can be terminated if the price exceeds the bid or if there is a capacity constraint. This does not meet the requirement of continuous processing without downtime.

- ☐ Use Reserved Instances exclusively to handle the maximum capacity required.

Using Reserved Instances exclusively for maximum capacity can be expensive, especially considering the intermittent nature of the traffic. This option may result in significant unused capacity during periods of low traffic, making it less cost-effective.

- ☐ Use Reserved Instances for the baseline capacity and use Spot Instances to handle additional capacity.

While this is cost-effective, the use of Spot Instances for additional capacity might introduce downtime when Spot Instances are terminated. The problem statement explicitly requires no downtime, making this option unsuitable.

- ☒ Use Reserved Instances for the baseline capacity and use On-Demand Instances to handle additional capacity.

This option provides a balance between cost and availability. Reserved Instances cover the baseline capacity, ensuring a cost-effective solution for the predictable workload. On-Demand Instances handle the unpredictable and intermittent traffic, guaranteeing no downtime as these instances are always available. This is the most cost-effective option that meets the 'no downtime' requirement.

Overall Explanation:

The most cost-effective solution that guarantees no downtime is to use Reserved Instances for the baseline capacity and On-Demand Instances for handling additional capacity during periods of high and unpredictable traffic. Spot Instances are cheaper but introduce the risk of downtime due to potential termination, while relying solely on Reserved Instances may lead to underutilized capacity and higher costs. Therefore, option D provides the best balance between cost and availability to fulfill the requirements.

[View Discussions](#)[Ask AI](#)

Question 32

Not Attempted

Security



A security team needs to restrict access to specific AWS services and actions across all accounts within their organization. The organization is managed using AWS Organizations. The solution must be scalable, easily maintainable, and centrally managed. What should a solutions architect implement to meet these requirements?

- ☐ Create individual IAM policies in each account to deny access to the services and actions.

This solution is not scalable or centrally managed, as it requires managing policies in each individual account.

- ☐ Create a centralized IAM role and assume this role from each account to enforce the restrictions.

While this provides some centralization, it relies on proper role assumption in each account and is not as effective as SCPs for preventing actions.

- ☐ Implement AWS Config rules to detect and remediate non-compliant configurations regarding service access.

AWS Config rules can detect and remediate, but they don't actively prevent actions in the same way as SCPs.

- ☒ Create a service control policy (SCP) in the root organizational unit (OU) to deny access to the specified services and actions.

SCPs provide centralized control over permissions within an organization, making them ideal for restricting access across multiple accounts. Applying it to the root OU ensures it affects all accounts.

Overall Explanation:

Service control policies (SCPs) are designed for centralized permission management within AWS Organizations. By creating an SCP in the root organizational unit, the security team can deny access to specific services and actions across all member accounts, fulfilling the requirements of scalability, maintainability, and central management.

[View Discussions](#)[Ask AI](#)

Question 33

Not Attempted

Security



A company's public web application, fronted by an Application Load Balancer (ALB), has been targeted by recent web attacks. The company wants to minimize the risk of Distributed Denial of Service (DDoS) attacks. What should a solutions architect implement to best protect the application from DDoS attacks?

⊗ Add an Amazon Inspector agent to the ALB.

Amazon Inspector is a security assessment service that helps identify vulnerabilities and security issues. It does not prevent or mitigate DDoS attacks.

⊗ Configure Amazon Macie to prevent attacks.

Amazon Macie uses machine learning to discover, classify, and protect sensitive data. It focuses on data security and protection, not DDoS prevention.

✓ Enable AWS Shield Advanced to prevent attacks.

AWS Shield Advanced is a managed DDoS protection service that provides enhanced DDoS protection capabilities, including proactive monitoring and automatic inline mitigations.

⊗ Configure Amazon GuardDuty to monitor the ALB.

Amazon GuardDuty is a threat detection service that analyzes events and network traffic to identify potential security threats. While it can detect DDoS attacks, it doesn't actively prevent or mitigate them like AWS Shield Advanced.

Overall Explanation:

To mitigate DDoS attacks, AWS Shield Advanced is the most appropriate solution. It provides enhanced DDoS protection, including proactive monitoring, automatic inline mitigations, and real-time attack reporting. Amazon Inspector focuses on vulnerability assessments, Amazon Macie focuses on data protection, and Amazon GuardDuty focuses on threat detection, but none directly prevent DDoS attacks as effectively as AWS Shield Advanced.

[View Discussions](#)[Ask AI](#)

Question 34

Not Attempted

Security



A company is deploying a new web application on two Amazon EC2 instances behind a load balancer. The application requires access to an Amazon S3 bucket for storing user-uploaded files. A solutions architect needs to implement a secure and scalable solution for granting the EC2 instances access to the S3 bucket. What is the MOST secure and recommended approach?

- ☒ Create an IAM role that grants access to the S3 bucket and attach the role to the EC2 instances.

This is the recommended approach. IAM roles provide temporary credentials to EC2 instances, allowing them to access S3 without storing long-term credentials on the instances.

- ☐ Create an IAM policy that grants access to the S3 bucket and attach the policy directly to the EC2 instances.

IAM policies cannot be directly attached to EC2 instances. They need to be attached to IAM users, groups or roles.

- ☐ Create an IAM group that grants access to the S3 bucket, add the EC2 instances to the group, and configure the group to allow access to S3.

IAM groups are designed for managing permissions for IAM users, not for granting permissions to EC2 instances. While you can indirectly manage permissions through a group if an IAM user is used, it's not the recommended pattern for services.

- ☐ Create IAM users for each EC2 instance, grant each user access to the S3 bucket, and configure the application to use the IAM user credentials.

This approach is not recommended because it involves managing and rotating long-term IAM user credentials on the EC2 instances, which increases the risk of credential compromise.

Overall Explanation:

IAM roles are the best practice for granting AWS services, like EC2 instances, access to other AWS resources, such as S3 buckets. By creating an IAM role with the necessary permissions and attaching it to the EC2 instances, the application can securely access the S3 bucket without the need to manage access keys directly on the instances. This approach promotes better security and easier management of permissions.

 View Discussions

 Ask AI

A company's web application is running on Amazon EC2 instances behind an Application Load Balancer. The company recently changed its policy, which now requires the application to be accessed from one specific country only. Which configuration will meet this requirement?

- ☐ **Configure the security group for the EC2 instances.**
Security groups control traffic at the instance level and do not provide country-specific filtering capabilities.
- ☐ **Configure the security group on the Application Load Balancer.**
Security groups control traffic at the instance level and do not provide country-specific filtering capabilities.
- ☒ **Configure AWS WAF on the Application Load Balancer in a VPC.**
AWS WAF allows you to create rules that include conditions based on the IP addresses' country of origin using the Geo Match feature, allowing you to restrict access to the application to a specific country.
- ☐ **Configure the network ACL for the subnet that contains the EC2 instances.**
Network ACLs control traffic at the subnet level based on IP addresses and port numbers, but they do not have built-in capabilities for country-based filtering.

Overall Explanation:

AWS WAF (Web Application Firewall) on the Application Load Balancer in a VPC allows you to restrict application access based on the geographic location of the viewers using Geo Match conditions. You can choose the countries from which AWS WAF should allow access. Security groups and Network ACLs do not have the ability to perform country-based filtering. Therefore, configuring AWS WAF is the correct approach.

 [View Discussions](#)

 [Ask AI](#)

Question 36

Not Attempted

[Compute](#)



A company provides an API to its users that automates inquiries for tax computations based on item prices. The company experiences a larger number of inquiries during the holiday season, resulting in slower response times. A solutions architect needs to design a solution that is scalable and elastic to address these performance issues. Which of the following architectural designs BEST meets these requirements?

- ⊗ Provide an API hosted on an Amazon EC2 instance. The EC2 instance performs the required computations when the API request is made.

This option is less scalable and elastic, as EC2 instances require manual scaling and management.

- ✔ Design a REST API using Amazon API Gateway that accepts the item names. API Gateway passes the item names to AWS Lambda for tax computations.

This option provides a scalable and elastic solution. API Gateway handles API requests, and Lambda automatically scales to handle varying workloads.

- ⊗ Create an Application Load Balancer that has two Amazon EC2 instances behind it. The EC2 instances will compute the tax on the received item names.

This option does not provide sufficient scalability and elasticity compared to a serverless solution. It also requires managing the EC2 instances.

- ⊗ Design a REST API using Amazon API Gateway that connects with an API hosted on an Amazon EC2 instance. API Gateway accepts and passes the item names to the EC2 instance for tax computations.

This option adds unnecessary complexity and is less scalable than using Lambda directly. It also requires managing an EC2 instance.

Overall Explanation:

The best solution is to use a serverless architecture with API Gateway and Lambda. API Gateway handles the API requests, and Lambda provides the necessary compute power, scaling automatically as needed. This approach eliminates the need to manage servers and ensures optimal performance even during peak seasons. Therefore, option B is the most scalable and elastic option.

 View Discussions

 Ask AI

Question 37

Not Attempted

Security



A solutions architect is creating a new Amazon CloudFront distribution for an application. Some of the information submitted by users is sensitive. The application uses HTTPS but needs another layer of security. The sensitive information should be protected throughout the entire application stack, and access to the information should be restricted to certain applications. Which action should the solutions architect take?

- ☐ **Configure a CloudFront signed URL.**
Signed URLs control access to specific resources but don't encrypt data within requests.
- ☐ **Configure a CloudFront signed cookie.**
Signed cookies control access but don't encrypt data within requests.
- ☒ **Configure a CloudFront field-level encryption profile.**
Field-level encryption encrypts specific data fields at the edge, ensuring protection throughout the application stack and restricting access to authorized applications.
- ☐ **Configure CloudFront and set the Origin Protocol Policy setting to HTTPS Only for the Viewer Protocol Policy.**
Setting the Origin Protocol Policy to HTTPS Only enforces secure communication but doesn't protect sensitive data within the application stack.

Overall Explanation:

CloudFront's field-level encryption allows encrypting specific data fields using encryption keys. The sensitive data is encrypted close to the user, and remains encrypted throughout your entire application stack. This encryption ensures that only applications that need the data—and have the credentials to decrypt it—are able to do so.

[View Discussions](#)[Ask AI](#)

Question 38

Not Attempted

Networking



A gaming company hosts a browser-based application on AWS. The application's users consume a large number of videos and images stored in Amazon S3. This content is the same for all users. The application has gained significant popularity, with millions of users worldwide accessing these media files. The company wants to efficiently deliver the files to users while minimizing the load on the origin server. Which solution is the MOST cost-effective?

- ☐ **Deploy an AWS Global Accelerator accelerator in front of the web servers.**
Global Accelerator is better suited for non-HTTP (TCP/UDP) traffic and provides static IP addresses. It is not the most cost-effective for serving static media files globally.
- ☒ **Deploy an Amazon CloudFront web distribution in front of the S3 bucket.**

CloudFront is a content delivery network (CDN) that caches content at edge locations, reducing latency and the load on the origin S3 bucket. It is designed for serving static and dynamic content globally.

- ⊗ Deploy an Amazon ElastiCache for Redis instance in front of the web servers.

ElastiCache for Redis is an in-memory data store typically used for caching database query results or session state, not for serving large media files directly.

- ⊗ Deploy an Amazon ElastiCache for Memcached instance in front of the web servers.

ElastiCache for Memcached is an in-memory cache often used to improve application performance by caching frequently accessed data. It is not designed for serving large media files at scale.

Overall Explanation:

Amazon CloudFront is the most cost-effective solution for delivering media files to millions of users worldwide while reducing the load on the origin. It leverages edge locations to cache the media content closer to the users, resulting in faster delivery and a better user experience. Global Accelerator is more suitable for TCP/UDP traffic, while ElastiCache is designed for caching database results and other application data.

 View Discussions

 Ask AI

Question 39

Not Attempted

Compute



A company has a multi-tier application running six front-end web servers in an Amazon EC2 Auto Scaling group within a single Availability Zone, fronted by an Application Load Balancer (ALB). A solutions architect aims to enhance the infrastructure for high availability without altering the application code. Which architectural change should the solutions architect implement to achieve this goal?

- ⊗ Create an Auto Scaling group that spans three instances across each of two AWS Regions.

Auto Scaling groups cannot span multiple AWS Regions. This option introduces complexity without directly addressing high availability within the existing application's region.

- ✓ Modify the Auto Scaling group to utilize three instances across each of two Availability Zones.

Distributing instances across multiple Availability Zones ensures that if one AZ experiences an outage, the application remains available in the other AZ, providing high availability.

- ⊗ Create an Auto Scaling template to facilitate quick instance creation in an alternate AWS Region.

While this prepares for disaster recovery, it doesn't directly address the immediate need for high availability within the current region without modifying the application.

- ⊗ Change the ALB's routing configuration to a round-robin setup, balancing traffic across the existing EC2 instances.

Round-robin load balancing alone does not guarantee high availability. If the single Availability Zone experiences an outage, all instances become unavailable regardless of the load balancing method.

Overall Explanation:

Modifying the Auto Scaling group to distribute instances across multiple Availability Zones is the correct solution. This provides redundancy and ensures that if one Availability Zone experiences an outage, the application remains available in the other AZ, fulfilling the requirement for high availability without modifying the application. Using multiple availability zones protects against availability zone level failures.

[View Discussions](#)[Ask AI](#)

Question 40

Not Attempted

Database



An e-commerce application uses Amazon API Gateway, AWS Lambda, and an Amazon Aurora PostgreSQL database for order processing. During a flash sale, customers experienced timeouts due to high CPU and memory utilization on the database caused by numerous open connections. A solutions architect needs to resolve the timeout errors with minimal application changes. Which solution meets these requirements?

- ⊗ Configure provisioned concurrency for the Lambda function. Modify the database to be a global database in multiple AWS Regions.

Provisioned concurrency doesn't directly address connection limits. Creating a global database introduces complexity without immediate benefit to resolving the timeout errors.

- ✓ Use Amazon RDS Proxy to create a proxy for the database. Modify the Lambda function to use the RDS Proxy endpoint instead of the database endpoint.
RDS Proxy pools and shares database connections, improving database efficiency and application scalability. It minimizes code changes by simply updating the Lambda function's endpoint.
- ✗ Create a read replica for the database in a different AWS Region. Use query string parameters in API Gateway to route traffic to the read replica.
Creating a read replica introduces data replication and management complexity, and query string parameters may not effectively distribute the write load causing the initial issue.
- ✗ Migrate the data from Aurora PostgreSQL to Amazon DynamoDB by using AWS Database Migration Service (AWS DMS). Modify the Lambda function to use the DynamoDB table.
Migrating to DynamoDB involves a significant change in data storage and requires extensive application modifications, conflicting with the requirement for minimal changes.

Overall Explanation:

Amazon RDS Proxy is the ideal solution. It acts as a connection pooler, reducing the number of open connections to the database and alleviating CPU and memory pressure during peak loads. Modifying the Lambda function to use the RDS Proxy endpoint is a minimal change that avoids major application refactoring. It addresses the root cause (too many connections) directly, preventing timeouts during sales events.

[View Discussions](#)[Ask AI](#)

Question 41

Not Attempted

Security



An application runs on Amazon EC2 instances deployed in private subnets within a VPC. This application requires secure access to an Amazon DynamoDB table. What is the MOST secure method to enable this access while ensuring that all traffic remains within the AWS network?

- ✓ Use a VPC endpoint for DynamoDB.
A VPC endpoint for DynamoDB enables instances in your VPC to access DynamoDB using private IP addresses without exposing the traffic to the public internet, ensuring that all traffic stays within the AWS network.
- ✗ Use a NAT gateway in a public subnet.

A NAT gateway requires an internet gateway and routes traffic through a public subnet, which means traffic would leave the AWS network.

⊗ Use a NAT instance in a private subnet.

A NAT instance, similar to a NAT gateway, also requires an internet gateway and exposes traffic outside the AWS network.

⊗ Use the internet gateway attached to the VPC.

Using the internet gateway forces traffic to traverse the public internet, which is not secure and does not keep traffic within the AWS network.

Overall Explanation:

The most secure method is to use a VPC endpoint for DynamoDB. A VPC endpoint allows private connectivity to DynamoDB without requiring an internet gateway, NAT device, or VPN connection, ensuring that all traffic remains within the AWS network. Other options either expose traffic to the internet or require public subnets, compromising security.

 View Discussions

 Ask AI

Question 42

Not Attempted

Database



An entertainment company is using Amazon DynamoDB to store media metadata. The application is read-intensive and experiencing performance delays. The company wants to improve the performance efficiency of DynamoDB without reconfiguring the application and with minimal operational overhead. Which solution should a solutions architect recommend?

⊗ Use Amazon ElastiCache for Redis.

Using Amazon ElastiCache for Redis would require modifying the application code and is not specifically designed to enhance DynamoDB performance.

✔ Use Amazon DynamoDB Accelerator (DAX).

Amazon DynamoDB Accelerator (DAX) is a fully managed, in-memory cache that improves read performance without application reconfiguration. It significantly reduces read latency and offloads traffic from DynamoDB tables.

- ☐ Replicate data by using DynamoDB global tables.
Replicating data with DynamoDB global tables would require additional configuration and operational overhead and may not address the read latency issues directly.
- ☐ Use Amazon ElastiCache for Memcached with Auto Discovery enabled.
Using Amazon ElastiCache for Memcached would require application code modifications and is not specifically designed for improving DynamoDB performance.

Overall Explanation:

Amazon DynamoDB Accelerator (DAX) is the recommended solution as it is purpose-built for enhancing DynamoDB performance without application reconfiguration. DAX provides a managed caching layer that significantly reduces read latency and offloads read traffic from DynamoDB tables. It is a fully managed, in-memory cache service.

[View Discussions](#)[Ask AI](#)

Question 43

Not Attempted

Database



A company's infrastructure consists of Amazon EC2 instances and an Amazon RDS DB instance in a single AWS Region. The company wants to back up its data in a separate Region. Which solution will meet these requirements with the LEAST operational overhead?

- ☒ Use AWS Backup to copy EC2 backups and RDS backups to the separate Region.
AWS Backup simplifies the backup process and automates the copying of backups to another Region, reducing manual effort. It supports both EC2 and RDS.
- ☐ Use Amazon Data Lifecycle Manager (Amazon DLM) to copy EC2 backups and RDS backups to the separate Region.
DLM is primarily designed for managing EBS snapshots lifecycle and doesn't directly handle cross-region RDS backups.
- ☐ Create Amazon Machine Images (AMIs) of the EC2 instances. Copy the AMIs to the separate Region. Create a read replica for the RDS DB instance in the separate Region.
Creating AMIs and RDS read replicas adds significant complexity and operational overhead compared to AWS Backup.

- ⊗ Create Amazon Elastic Block Store (Amazon EBS) snapshots. Copy the EBS snapshots to the separate Region. Create RDS snapshots. Export the RDS snapshots to Amazon S3. Configure S3 Cross-Region Replication (CRR) to the separate Region.

This option involves multiple manual steps and configurations, making it more complex and increasing operational overhead.

Overall Explanation:

AWS Backup provides a fully managed, centralized backup service across AWS services and supports cross-region backups with minimal operational overhead. The other options involve more manual steps and configurations. DLM does not support RDS, and manually creating AMIs/Snapshots and Read Replicas is more complex.

[View Discussions](#)[Ask AI](#)

Question 44

Not Attempted

Security



A solutions architect needs to securely store a database user name and password that an application uses to access an Amazon RDS DB instance. The application that accesses the database runs on an Amazon EC2 instance. The solutions architect wants to create a secure parameter in AWS Systems Manager Parameter Store. What should the solutions architect do to meet this requirement?

- ✔ Create an IAM role that has read access to the Parameter Store parameter. Allow Decrypt access to an AWS Key Management Service (AWS KMS) key that is used to encrypt the parameter. Assign this IAM role to the EC2 instance.
This is the correct approach. IAM roles are used to grant permissions to AWS services like EC2 instances. This allows the EC2 instance to retrieve the encrypted password and decrypt it using KMS.
- ⊗ Create an IAM policy that allows read access to the Parameter Store parameter. Allow Decrypt access to an AWS Key Management Service (AWS KMS) key that is used to encrypt the parameter. Assign this IAM policy to the EC2 instance.
IAM policies are typically attached to users or groups, not directly to EC2 instances. IAM roles are designed for services like EC2.
- ⊗ Create an IAM trust relationship between the Parameter Store parameter and the EC2 instance. Specify Amazon RDS as a principal in the trust policy.

IAM trust relationships are used for cross-account access or allowing services to assume roles. This is not the correct approach for granting an EC2 instance access to a Parameter Store parameter.

- ⊗ Create an IAM trust relationship between the DB instance and the EC2 instance. Specify Systems Manager as a principal in the trust policy.

IAM trust relationships are used for cross-account access or allowing services to assume roles. The DB instance doesn't need to trust the EC2 instance in this scenario. Specifying Systems Manager as the principal doesn't enable the EC2 instance to retrieve the password.

Overall Explanation:

The correct approach is to create an IAM role with read access to the Parameter Store parameter and Decrypt access to the KMS key used to encrypt the parameter. Assigning this role to the EC2 instance allows the application to securely retrieve and decrypt the database credentials. IAM roles are the correct mechanism for assigning permissions to EC2 instances. A policy is not the right choice and trust relationships are for different use cases.

 View Discussions

 Ask AI

Question 45

Not Attempted

Compute



An application development team is designing a microservice that will convert large images to smaller, compressed images. When a user uploads an image through the web interface, the microservice should store the image in an Amazon S3 bucket, process and compress the image with an AWS Lambda function, and store the image in its compressed form in a different S3 bucket. A solutions architect needs to design a solution that uses durable, stateless components to process the images automatically. Which combination of actions will meet these requirements?

- ✔ Create an Amazon Simple Queue Service (Amazon SQS) queue. Configure the S3 bucket to send a notification to the SQS queue when an image is uploaded to the S3 bucket.

Configuring S3 to send notifications to an SQS queue provides durability and decouples the upload process from the processing, ensuring reliable event capture.

- ✔ Configure the Lambda function to use the Amazon Simple Queue Service (Amazon SQS) queue as the invocation source. When the SQS message is successfully processed,

delete the message in the queue.

Setting SQS as a Lambda invocation source ensures stateless processing; Lambda is triggered by messages in the queue, and deleting messages after processing avoids duplicate executions.

- ⊗ Configure the Lambda function to monitor the S3 bucket for new uploads. When an uploaded image is detected, write the file name to a text file in memory and use the text file to keep track of the images that were processed.

Storing processing state within the Lambda function's memory is not durable or scalable. Lambda functions are stateless and may be invoked on different infrastructure for each event.

- ⊗ Launch an Amazon EC2 instance to monitor an Amazon Simple Queue Service (Amazon SQS) queue. When items are added to the queue, log the file name in a text file on the EC2 instance and invoke the Lambda function.

Using an EC2 instance to monitor an SQS queue and trigger the Lambda function adds unnecessary complexity. SQS can directly trigger Lambda.

- ⊗ Configure an Amazon EventBridge (Amazon CloudWatch Events) event to monitor the S3 bucket. When an image is uploaded, send an alert to an Amazon Simple Notification Service (Amazon SNS) topic with the application owner's email address for further processing.

While EventBridge and SNS can be used for notifications, this configuration requires manual intervention and does not automate the image processing as required.

Overall Explanation:

The combination of creating an SQS queue and configuring the S3 bucket to send notifications to it (Option A), along with configuring the Lambda function to be triggered by the SQS queue (Option B), provides a durable, stateless, and automated image processing pipeline. SQS decouples the S3 upload event from the image processing Lambda, ensuring that events are not lost and Lambda processes them in a stateless manner.

 View Discussions

 Ask AI

Question 46

Not Attempted

Security



A company is designing a cloud communications platform that is driven by APIs. The application is hosted on Amazon EC2 instances behind a Network Load Balancer (NLB). The

company uses Amazon API Gateway to provide external users with access to the application through APIs. The company wants to protect the platform against web exploits like SQL injection and also wants to detect and mitigate large, sophisticated DDoS attacks. Which combination of solutions provides the MOST protection? (Choose two.)

☐ Use AWS WAF to protect the NLB.

AWS WAF is not compatible with Network Load Balancers. It works with Application Load Balancers, API Gateway, and CloudFront.

☒ Use AWS Shield Advanced with the NLB.

AWS Shield Advanced provides enhanced DDoS protection for resources like NLBs.

☒ Use AWS WAF to protect Amazon API Gateway.

AWS WAF protects API Gateway against web exploits like SQL injection by filtering malicious traffic.

☐ Use Amazon GuardDuty with AWS Shield Standard

GuardDuty detects threats but doesn't directly protect against exploits or mitigate sophisticated DDoS attacks. Shield Standard offers basic protection but doesn't target NLBs specifically.

☐ Use AWS Shield Standard with Amazon API Gateway.

Shield Standard provides basic DDoS protection, but doesn't address web exploits or the need for sophisticated DDoS mitigation.

Overall Explanation:

The best combination is to use AWS Shield Advanced with the NLB for sophisticated DDoS protection and AWS WAF to protect Amazon API Gateway against web exploits like SQL injection. WAF operates at Layer 7 and protects application layer, which API Gateway requires. AWS Shield Advanced handles Layer 3/4 DDoS protection for the NLB.

 View Discussions

 Ask AI

Question 47

Not Attempted

Compute



A company has a legacy data processing application that runs on Amazon EC2 instances. Data is processed sequentially, but the order of results does not matter. The application uses a monolithic architecture. The only way that the company can scale the application to meet

increased demand is to increase the size of the instances. The company's developers have decided to rewrite the application to use a microservices architecture on Amazon Elastic Container Service (Amazon ECS). What should a solutions architect recommend for communication between the microservices?

- ☒ Create an Amazon Simple Queue Service (Amazon SQS) queue. Add code to the data producers, and send data to the queue. Add code to the data consumers to process data from the queue.

This is the best option. SQS allows for asynchronous communication between microservices, decoupling the data producers and consumers. It provides scalability and ensures data processing can happen independently. Since order doesn't matter, a standard queue is appropriate.
- ☐ Create an Amazon Simple Notification Service (Amazon SNS) topic. Add code to the data producers, and publish notifications to the topic. Add code to the data consumers to subscribe to the topic.

SNS is more suitable for pub/sub messaging, where multiple subscribers receive the same message. It is not the best fit for sequential data processing between specific microservices.
- ☐ Create an AWS Lambda function to pass messages. Add code to the data producers to call the Lambda function with a data object. Add code to the data consumers to receive a data object that is passed from the Lambda function.

Using AWS Lambda functions for communication introduces unnecessary complexity and is not the optimal solution for sequential data processing or decoupling. It creates tight coupling between producers and consumers.
- ☐ Create an Amazon DynamoDB table. Enable DynamoDB Streams. Add code to the data producers to insert data into the table. Add code to the data consumers to use the DynamoDB Streams API to detect new table entries and retrieve the data.

DynamoDB with DynamoDB Streams is primarily designed for real-time data streaming and change capture scenarios. It is not the most efficient choice for sequential data processing in a microservices architecture.

Overall Explanation:

The correct answer is A. Creating an Amazon SQS queue decouples the microservices and allows for asynchronous processing. The data producers send messages to the queue, and the data consumers independently process messages from the queue. This approach provides scalability, flexibility, and resilience. Since the order of results doesn't matter, a

standard SQS queue is suitable. Options B, C, and D introduce unnecessary complexity or are not designed for this type of sequential, decoupled processing.

[View Discussions](#)[Ask AI](#)

Question 48

Not Attempted

Database



A company is migrating its on-premises MySQL database to AWS. They recently experienced a significant database outage and require a reliable AWS database solution that minimizes data loss and ensures every transaction is stored on at least two nodes. Which of the following options best meets these requirements?

- ☐ Create an Amazon RDS DB instance with synchronous replication to three nodes in three Availability Zones.

While this offers high availability, Amazon RDS does not natively support synchronous replication to three nodes across three Availability Zones.

- ☒ Create an Amazon RDS MySQL DB instance with Multi-AZ functionality enabled to synchronously replicate the data.

Enabling Multi-AZ in RDS provides synchronous replication to a standby instance, ensuring minimal data loss and storing data on at least two nodes. This is the correct answer.

- ☐ Create an Amazon RDS MySQL DB instance and then create a read replica in a separate AWS Region that synchronously replicates the data.

Read replicas in RDS use asynchronous replication, which does not guarantee minimal data loss in case of a primary instance failure.

- ☐ Create an Amazon EC2 instance with a MySQL engine installed that triggers an AWS Lambda function to synchronously replicate the data to an Amazon RDS MySQL DB instance.

This solution introduces unnecessary complexity. RDS Multi-AZ provides a managed and more reliable solution for synchronous replication.

Overall Explanation:

Amazon RDS Multi-AZ functionality provides synchronous replication to a standby instance in a different Availability Zone, minimizing data loss and fulfilling the requirement of storing data on at least two nodes. Read replicas use asynchronous replication, and

manually managing replication using EC2 and Lambda adds unnecessary complexity. RDS does not support synchronous replication to three nodes.

[View Discussions](#)[Ask AI](#)

Question 49

Not Attempted

Database



A company is building a new dynamic ordering website. The company wants to minimize server maintenance and patching. The website must be highly available and must scale read and write capacity as quickly as possible to meet changes in user demand. Which solution will meet these requirements?

- ☒ Host static content in Amazon S3. Host dynamic content by using Amazon API Gateway and AWS Lambda. Use Amazon DynamoDB with on-demand capacity for the database. Configure Amazon CloudFront to deliver the website content.

This option leverages serverless technologies (S3, API Gateway, Lambda, DynamoDB) to minimize maintenance and patching. DynamoDB with on-demand capacity provides automatic scaling of read/write capacity, ensuring high availability and responsiveness to changes in user demand. This is ideal for OLTP use cases.

- ☐ Host static content in Amazon S3. Host dynamic content by using Amazon API Gateway and AWS Lambda. Use Amazon Aurora with Aurora Auto Scaling for the database. Configure Amazon CloudFront to deliver the website content.

While this option uses serverless components for static and dynamic content, Aurora Auto Scaling primarily focuses on scaling read replicas. It does not provide the same level of automatic scaling for write capacity as DynamoDB with on-demand capacity.

- ☐ Host all the website content on Amazon EC2 instances. Create an Auto Scaling group to scale the EC2 instances. Use an Application Load Balancer to distribute traffic. Use Amazon DynamoDB with provisioned write capacity for the database.

Using EC2 instances requires more server maintenance and patching compared to serverless options. While Auto Scaling helps with scaling, it's not as seamless and maintenance-free as Lambda and API Gateway. Additionally, provisioned write capacity requires manual adjustments.

- ☐ Host all the website content on Amazon EC2 instances. Create an Auto Scaling group to scale the EC2 instances. Use an Application Load Balancer to distribute traffic. Use Amazon Aurora with Aurora Auto Scaling for the database.

Similar to option C, using EC2 instances introduces more maintenance overhead. Also, Aurora Auto Scaling primarily scales read replicas, which doesn't address the requirement for quickly scaling

write capacity.

Overall Explanation:

The best solution is to use a combination of serverless technologies. Option A utilizes Amazon S3 for static content, API Gateway and Lambda for dynamic content, and DynamoDB with on-demand capacity for the database. This minimizes server maintenance, provides high availability, and allows for rapid scaling of both read and write capacity to meet changing user demands. DynamoDB is particularly well-suited for OLTP workloads and provides automatic scaling, while Aurora Auto Scaling primarily focuses on read replica scaling. The EC2-based solutions require more manual maintenance and do not scale as efficiently as serverless options.

[View Discussions](#)[Ask AI](#)

Question 50

Not Attempted

Networking



A company has an AWS account used for software engineering. The AWS account has access to the company's on-premises data center through a pair of AWS Direct Connect connections. All non-VPC traffic routes to the virtual private gateway. A development team recently created an AWS Lambda function through the console. The development team needs to allow the function to access a database that runs in a private subnet in the company's data center. Which solution will meet these requirements?

- ☒ **Configure the Lambda function to run in the VPC with the appropriate security group.**
Configuring the Lambda function to run within the VPC and assigning it the appropriate security group allows it to access resources within the VPC, including the connection to the on-premises data center. The security group will need to be configured to allow outbound traffic to the database.
- ☐ **Set up a VPN connection from AWS to the data center. Route the traffic from the Lambda function through the VPN.**
Setting up a VPN is unnecessary. The company already has Direct Connect, which provides a dedicated network connection. Adding a VPN adds complexity and is redundant.
- ☐ **Update the route tables in the VPC to allow the Lambda function to access the on-premises data center through Direct Connect.**
The route tables are likely already configured to route traffic to the on-premises data center via the virtual private gateway since 'All non-VPC traffic routes to the virtual private gateway'. The key is to

associate the Lambda function with the VPC and use security groups for access control.

- ⊗ Create an Elastic IP address. Configure the Lambda function to send traffic through the Elastic IP address without an elastic network interface.

Lambda functions cannot directly utilize Elastic IP addresses without an Elastic Network Interface (ENI). This configuration is invalid and won't enable access to the on-premises database.

Overall Explanation:

The correct answer is to configure the Lambda function to run in the VPC with the appropriate security group. This allows the function to leverage the existing Direct Connect connection to access the on-premises database, while the security group provides the necessary access control. Since 'All non-VPC traffic routes to the virtual private gateway' all routes are already configured to the on-premise location.

 View Discussions

 Ask AI

Question 51

Not Attempted

Security



A company runs an application using Amazon ECS. The application creates resized versions of an original image and then makes Amazon S3 API calls to store the resized images in Amazon S3. How can a solutions architect ensure that the application has permission to access Amazon S3?

- ⊗ Update the S3 role in AWS IAM to allow read/write access from Amazon ECS, and then relaunch the container.

Updating the S3 role in IAM and relaunching the container does not associate the updated role with the ECS task.

- ✓ Create an IAM role with S3 permissions, and then specify that role as the taskRoleArn in the task definition.

This approach allows the ECS task to assume the specified role and gain the necessary permissions to access Amazon S3.

- ⊗ Create a security group that allows access from Amazon ECS to Amazon S3, and update the launch configuration used by the ECS cluster.

Creating a security group that allows access from Amazon ECS to Amazon S3 does not grant the necessary permissions to the ECS task.

- ⊗ Create an IAM user with S3 permissions, and then relaunch the Amazon EC2 instances for the ECS cluster while logged in as this account.

Creating an IAM user with S3 permissions and relaunching the EC2 instances for the ECS cluster does not associate the IAM user with the ECS task.

Overall Explanation:

The correct way to grant an ECS application permission to access S3 is to create an IAM role with the necessary S3 permissions and specify that role as the `taskRoleArn` in the task definition. This allows the ECS task to assume the specified role and gain the necessary permissions to access Amazon S3.

 View Discussions

 Ask AI

Question 52

Not Attempted

Storage



A company is migrating a Windows-based application to AWS. The application requires a shared Windows file system accessible by multiple Amazon EC2 Windows instances across different Availability Zones. Which solution should a solutions architect implement to meet this requirement?

- ⊗ Configure AWS Storage Gateway in volume gateway mode. Mount the volume to each Windows instance.

AWS Storage Gateway in volume gateway mode is not designed for providing a shared file system. It primarily focuses on integrating on-premises storage with AWS.

- ✔ Configure Amazon FSx for Windows File Server. Mount the Amazon FSx file system to each Windows instance.

Amazon FSx for Windows File Server is a fully managed service that provides a native Windows file system accessible over the SMB protocol, making it suitable for Windows-based applications needing a shared file system.

- ⊗ Configure a file system by using Amazon Elastic File System (Amazon EFS). Mount the EFS file system to each Windows instance.

Amazon EFS is designed for Linux-based workloads using NFS and is not natively compatible with Windows file systems.

- ☐ Configure an Amazon Elastic Block Store (Amazon EBS) volume with the required size. Attach each EC2 instance to the volume. Mount the file system within the volume to each Windows instance.

An Amazon EBS volume cannot be attached to multiple EC2 instances simultaneously, making it unsuitable for creating a shared file system.

Overall Explanation:

Amazon FSx for Windows File Server is the appropriate solution because it provides a fully managed, native Windows file system (SMB) suitable for sharing data between multiple Windows EC2 instances across Availability Zones. EFS is for Linux, EBS cannot be attached to multiple instances simultaneously, and Storage Gateway isn't a file storage service. Therefore, FSx is the correct choice.

 View Discussions

 Ask AI

Question 53

Not Attempted

Database



A company is developing an e-commerce application with a load-balanced front end, a container-based application, and a relational database. The Solutions Architect needs a highly available solution with minimal manual intervention. Which solutions meet these requirements? (Choose two)

- ☒ Create an Amazon RDS DB instance in Multi-AZ mode.
Multi-AZ RDS provides high availability with automatic failover, minimizing manual intervention.
- ☐ Create an Amazon RDS DB instance and one or more read replicas in another Availability Zone.
While read replicas enhance read performance, they don't provide automatic failover like Multi-AZ deployments and require manual intervention to promote a replica to a primary instance.
- ☐ Create an Amazon EC2 instance-based Docker cluster to handle the dynamic application load.
Managing a Docker cluster on EC2 requires manual intervention for scaling, patching, and maintenance, increasing operational overhead.

- ☒ Create an Amazon Elastic Container Service (Amazon ECS) cluster with a Fargate launch type to handle the dynamic application load.
- ECS with Fargate removes the need to manage underlying infrastructure, providing automatic scaling and high availability with minimal manual intervention.
- ☐ Create an Amazon Elastic Container Service (Amazon ECS) cluster with an Amazon EC2 launch type to handle the dynamic application load.
- Using ECS with EC2 requires managing the EC2 instances, including scaling, patching, and ensuring availability, increasing manual intervention.

Overall Explanation:

The best solutions for high availability and minimal manual intervention are: * **Amazon RDS in Multi-AZ mode:** This ensures automatic failover to a standby replica in another Availability Zone, providing high availability for the database tier. * **Amazon ECS with Fargate:** Fargate abstracts the underlying infrastructure, automatically scaling and managing the containers, making it a highly available and low-maintenance option for the containerized application.

[View Discussions](#)[Ask AI](#)

Question 54

Not Attempted

Storage



A company uses Amazon S3 as its data lake. The company has a new partner that must use SFTP to upload data files. A solutions architect needs to implement a highly available SFTP solution that minimizes operational overhead. Which solution will meet these requirements?

- ☒ Use AWS Transfer Family to configure an SFTP-enabled server with a publicly accessible endpoint. Choose the S3 data lake as the destination.
- AWS Transfer Family provides a fully managed SFTP service that can integrate directly with S3. It handles scaling, availability, and security automatically with minimal overhead.
- ☐ Use Amazon S3 File Gateway as an SFTP server. Expose the S3 File Gateway endpoint URL to the new partner. Share the S3 File Gateway endpoint with the new partner.
- Amazon S3 File Gateway is primarily used for file-based access to S3 storage over NFS or SMB protocols, not for SFTP access.

- ⊗ Launch an Amazon EC2 instance in a private subnet in a VPC. Instruct the new partner to upload files to the EC2 instance by using a VPN. Run a cron job script on the EC2 instance to upload files to the S3 data lake.

This requires manual management of an EC2 instance, VPN setup, and a cron job script for uploading files, which introduces operational overhead and complexity.

- ⊗ Launch Amazon EC2 instances in a private subnet in a VPC. Place a Network Load Balancer (NLB) in front of the EC2 instances. Create an SFTP listener port for the NLB. Share the NLB hostname with the new partner. Run a cron job script on the EC2 instances to upload files to the S3 data lake.

This also requires manual management of EC2 instances, a Network Load Balancer, and cron job scripts for file uploads. It is more complex and involves additional components compared to AWS Transfer Family.

Overall Explanation:

AWS Transfer Family provides a fully managed SFTP service that can integrate directly with S3, handling scaling, availability, and security automatically with minimal overhead. The other options require more manual management and operational overhead. AWS Transfer Family securely scales your recurring business-to-business file transfers to AWS Storage services using SFTP, FTPS, FTP, and AS2 protocols.

 View Discussions

 Ask AI

Question 55

Not Attempted

Storage



A company needs to store contract documents. A contract lasts for 5 years. During the 5-year period, the company must ensure that the documents cannot be overwritten or deleted. The company needs to encrypt the documents at rest and rotate the encryption keys automatically every year. Which combination of steps should a solutions architect take to meet these requirements with the LEAST operational overhead?

- ⊗ Store the documents in Amazon S3. Use S3 Object Lock in governance mode.
S3 Object Lock in governance mode does not provide the required immutability, as it allows exceptions for privileged users.
- ✓ Store the documents in Amazon S3. Use S3 Object Lock in compliance mode.

S3 Object Lock in compliance mode enforces a strict retention policy, preventing any modifications or deletions.

- ⊗ Use server-side encryption with Amazon S3 managed encryption keys (SSE-S3).
Configure key rotation.

While SSE-S3 encrypts the data at rest, you cannot configure the key rotation frequency. The keys are rotated automatically by AWS.

- ✔ Use server-side encryption with AWS Key Management Service (AWS KMS) customer managed keys. Configure key rotation.

Using KMS customer managed keys allows you to encrypt data at rest and configure automatic key rotation on an annual basis.

- ⊗ Use server-side encryption with AWS Key Management Service (AWS KMS) customer provided (imported) keys. Configure key rotation.

Customer provided keys cannot be automatically rotated. The customer would have to manually provide/import new keys.

Overall Explanation:

To meet the requirements, the solutions architect should use S3 Object Lock in compliance mode to prevent overwrites or deletions for 5 years. They should also use server-side encryption with AWS KMS customer managed keys to encrypt the documents and automatically rotate the encryption keys every year. Compliance mode ensures immutability, and KMS allows for configurable key rotation.

[View Discussions](#)[Ask AI](#)

Question 56

Not Attempted

Networking



A company hosts a three-tier web application on AWS. The web servers reside in a public subnet within a VPC, while the application and database servers are in private subnets within the same VPC. A third-party virtual firewall appliance, obtained from AWS Marketplace, is deployed in a dedicated inspection VPC and configured to accept IP packets. The solutions architect needs to integrate the appliance for inspecting all traffic before it reaches the web servers, while minimizing operational overhead. Which solution is best?

- ⊗ Create a Network Load Balancer in the application's VPC public subnet to route traffic to the appliance for inspection.

Network Load Balancers are not designed to directly route traffic to third-party virtual firewall appliances for deep packet inspection.

- ⊗ Create an Application Load Balancer in the application's VPC public subnet to route traffic to the appliance for inspection.

Application Load Balancers operate at Layer 7 and cannot directly forward traffic to third-party appliances for network-level inspection.

- ⊗ Deploy a transit gateway in the inspection VPC. Configure route tables to route the incoming packets through the transit gateway.

While a Transit Gateway can connect VPCs, it doesn't inherently provide a mechanism for routing all traffic through a third-party firewall appliance with minimal operational overhead.

- ✓ Deploy a Gateway Load Balancer in the inspection VPC. Create a Gateway Load Balancer endpoint to receive incoming packets and forward them to the appliance.

Gateway Load Balancers are specifically designed for integrating third-party virtual appliances like firewalls. They provide a scalable and managed solution for intercepting and inspecting network traffic.

Overall Explanation:

A Gateway Load Balancer (GWLB) is the best choice. It is designed to make it easy to deploy, scale, and manage third-party virtual appliances, such as security appliances. The GWLB resides in the inspection VPC and uses a Gateway Load Balancer endpoint to seamlessly receive incoming traffic, forward it to the firewall appliance, and then route it to the destination. This provides centralized traffic inspection with the least operational overhead.

 View Discussions

 Ask AI

Question 57

Not Attempted

Compute



A company has a web application that is based on Java and PHP. The company plans to migrate the application from on premises to AWS. The company needs the ability to test new site features frequently. The company also requires a highly available and managed solution that demands minimal operational overhead. Which solution will meet these requirements?

- ☐ Create an Amazon S3 bucket. Enable static web hosting on the S3 bucket. Upload the static content to the S3 bucket. Use AWS Lambda to process all dynamic content.
This solution is best suited for static websites with limited dynamic functionality. It does not provide the necessary flexibility and dynamic functionality required by a Java and PHP-based web application.
- ☒ Deploy the web application to an AWS Elastic Beanstalk environment. Use URL swapping to switch between multiple Elastic Beanstalk environments for feature testing.
AWS Elastic Beanstalk is a fully managed service ideal for deploying and running web applications. URL swapping facilitates seamless switching between environments for testing new features with minimal downtime.
- ☐ Deploy the web application to Amazon EC2 instances that are configured with Java and PHP. Use Auto Scaling groups and an Application Load Balancer to manage the website's availability.
This approach requires manual management of EC2 instances, Auto Scaling groups, and the Application Load Balancer, leading to increased operational overhead and complexity.
- ☐ Containerize the web application. Deploy the web application to Amazon EC2 instances. Use the AWS Load Balancer Controller to dynamically route traffic between containers that contain the new site features for testing.
Containerization introduces additional complexity in managing containers and infrastructure, increasing the operational overhead.

Overall Explanation:

AWS Elastic Beanstalk provides a fully managed platform to deploy the web application and handle infrastructure management. URL swapping allows easy testing of new features before swapping into production, which results in minimal overhead compared to managing infrastructure directly.

[View Discussions](#)[Ask AI](#)

Question 58

Not Attempted

Database



A company has an ordering application that stores customer information in Amazon RDS for MySQL. During regular business hours, employees run one-time queries for reporting purposes. Timeouts are occurring during order processing because the reporting queries are taking a

long time to run. The company needs to eliminate the timeouts without preventing employees from performing queries. What should a solutions architect do to meet these requirements?

- ☒ Create a read replica. Move reporting queries to the read replica.
Moving reporting queries to a read replica isolates the read-heavy reporting workload from the primary database, preventing performance degradation and timeouts for the ordering application.
- ☐ Create a read replica. Distribute the ordering application to the primary DB instance and the read replica.
Ordering applications require write access, which a read replica cannot provide. Distributing the application in this manner is not feasible.
- ☐ Migrate the ordering application to Amazon DynamoDB with on-demand capacity.
Migrating to DynamoDB is a significant change involving schema and application modifications and it is not necessary to solve this particular problem. Also, it doesn't address the need to allow employees to perform reporting queries.
- ☐ Schedule the reporting queries for non-peak hours.
Scheduling for non-peak hours is not practical because the requirement states the queries need to run during regular business hours.

Overall Explanation:

The correct solution is to create a read replica and move the reporting queries to it. This offloads the read-heavy workload from the primary database instance, which is responsible for order processing. This separation ensures that the primary database can handle transactional operations efficiently, preventing timeouts and performance issues. Read replicas are designed for read-only operations like reporting, and they don't interfere with the primary database's write operations. Additionally, it allows employees to continue running queries during business hours.

[View Discussions](#)[Ask AI](#)

Question 59

Not Attempted

Database



A hospital wants to create digital copies for its large collection of historical written records. The hospital will continue to add hundreds of new documents each day. The hospital's data team will scan the documents and will upload the documents to the AWS Cloud. A solutions architect

must implement a solution to analyze the documents, extract the medical information, and store the documents so that an application can run SQL queries on the data. The solution must maximize scalability and operational efficiency. Which combination of steps should the solutions architect take to meet these requirements?

- ⊗ Write the document information to an Amazon EC2 instance that runs a MySQL database.

Writing document information directly to an EC2 instance with MySQL is not scalable or operationally efficient for this scenario.

- ✔ Write the document information to an Amazon S3 bucket. Use Amazon Athena to query the data.

Storing documents in S3 provides scalable and durable storage, and using Athena allows querying the data with SQL directly without loading into a database.

- ⊗ Create an Auto Scaling group of Amazon EC2 instances to run a custom application that processes the scanned files and extracts the medical information.

Creating an Auto Scaling group of EC2 instances for processing introduces unnecessary complexity and operational overhead.

- ⊗ Create an AWS Lambda function that runs when new documents are uploaded. Use Amazon Rekognition to convert the documents to raw text. Use Amazon Transcribe Medical to detect and extract relevant medical information from the text.

Transcribe Medical is designed for speech-to-text, not for extracting text from scanned documents. Rekognition is primarily for image identification and object detection, not document text extraction.

- ✔ Create an AWS Lambda function that runs when new documents are uploaded. Use Amazon Textract to convert the documents to raw text. Use Amazon Comprehend Medical to detect and extract relevant medical information from the text.

Using Lambda, Textract, and Comprehend Medical provides a scalable, serverless approach to automatically process documents, extract text, and identify medical information.

Overall Explanation:

Storing the documents in S3 and querying with Athena provides a scalable and cost-effective solution for running SQL queries on the data. Using a Lambda function triggered by uploads, along with Textract for text extraction and Comprehend Medical for medical information detection, allows for automated document analysis with minimal operational overhead. Therefore, options B and E are the correct combination of steps.

[View Discussions](#)[Ask AI](#)

Question 60

Not Attempted

Database



A company is running a batch application on Amazon EC2 instances. The application heavily relies on multiple Amazon RDS databases, leading to a high volume of read operations on these databases. A solutions architect needs to implement a solution to reduce the number of database reads while maintaining high availability for the application. What should the solutions architect do to meet this requirement?

☒ Add Amazon RDS read replicas.

RDS read replicas offload read traffic from the primary database, reducing its load and improving availability by providing a failover target. This directly addresses the requirement to reduce reads and ensure high availability.

☐ Use Amazon ElastiCache for Redis.

While ElastiCache for Redis can reduce database reads by caching frequently accessed data, it's primarily a caching solution and not a direct replacement for read replicas in the context of high availability for the database layer itself. It addresses reducing reads but not specifically HA for the RDS databases.

☐ Use Amazon Route 53 DNS caching.

Route 53 DNS caching helps reduce the number of DNS queries, which is unrelated to the database read operations and high availability requirements of the application's database layer.

☐ Use Amazon ElastiCache for Memcached.

ElastiCache for Memcached can reduce database reads, but it does not inherently offer the same level of high availability features as RDS read replicas (for the database itself) or Redis (for caching).

Overall Explanation:

The best solution is to add Amazon RDS read replicas. Read replicas offload read traffic from the primary database, reducing the load and directly addressing the requirement to reduce database reads. Furthermore, read replicas enhance high availability by providing a failover target in case the primary database instance becomes unavailable. ElastiCache can reduce reads, but read replicas provide HA for the database layer itself.

[View Discussions](#)[Ask AI](#)

Question 61

Not Attempted

Database



A company needs to run a critical application on AWS. The company needs to use Amazon EC2 for the application's database. The database must be highly available and must fail over automatically if a disruptive event occurs. Which solution will meet these requirements?

- ☒ Launch two EC2 instances, each in a different Availability Zone in the same AWS Region. Install the database on both EC2 instances. Configure the EC2 instances as a cluster. Set up database replication.

This option provides high availability by deploying instances in different Availability Zones and setting up a database cluster with replication for automatic failover.

- ☐ Launch an EC2 instance in an Availability Zone. Install the database on the EC2 instance. Use an Amazon Machine Image (AMI) to back up the data. Use AWS CloudFormation to automate provisioning of the EC2 instance if a disruptive event occurs.

This option uses a single instance, and restoring from an AMI is not a highly available solution with automatic failover.

- ☐ Launch two EC2 instances, each in a different AWS Region. Install the database on both EC2 instances. Set up database replication. Fail over the database to a second Region.

While this provides disaster recovery, it's not necessary for high availability within a region and might involve manual failover, which doesn't meet the automatic failover requirement.

- ☐ Launch an EC2 instance in an Availability Zone. Install the database on the EC2 instance. Use an Amazon Machine Image (AMI) to back up the data. Use EC2 automatic recovery to recover the instance if a disruptive event occurs.

This option uses a single instance, and EC2 automatic recovery does not provide the database-level high availability and automatic failover required.

Overall Explanation:

Option A is the correct solution because it provides high availability within the same AWS Region using multiple Availability Zones and database clustering. This allows for automatic failover in the event of a disruptive event. While option C provides disaster recovery by utilizing multiple regions, it goes beyond the stated HA requirement, and failover might not be automated. Options B and D rely on single instances and are not highly available.

[View Discussions](#)[Ask AI](#)

Question 62

Not Attempted

Automation



A company's order system sends requests from clients to Amazon EC2 instances. The EC2 instances process the orders and then store the orders in a database on Amazon RDS. Users report that they must reprocess orders when the system fails. The company wants a resilient solution that can process orders automatically if a system outage occurs. What should a solutions architect do to meet these requirements?

- ☐ Move the EC2 instances into an Auto Scaling group. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to target an Amazon Elastic Container Service (Amazon ECS) task.

This option does not provide the necessary decoupling and message queuing for automatic order processing during outages.

- ☐ Move the EC2 instances into an Auto Scaling group behind an Application Load Balancer (ALB). Update the order system to send messages to the ALB endpoint.

This option does not address the need for message queuing and automatic processing during outages.

- ☒ Move the EC2 instances into an Auto Scaling group. Configure the order system to send messages to an Amazon Simple Queue Service (Amazon SQS) queue. Configure the EC2 instances to consume messages from the queue.

This option provides a resilient solution by decoupling the order system from the processing system using SQS. The Auto Scaling group ensures high availability of EC2 instances to process the messages.

- ☐ Create an Amazon Simple Notification Service (Amazon SNS) topic. Create an AWS Lambda function, and subscribe the function to the SNS topic. Configure the order system to send messages to the SNS topic. Send a command to the EC2 instances to process the messages by using AWS Systems Manager Run Command.

This option relies on SNS and Lambda, which can be part of an event-driven architecture but may not be the best fit for ensuring the automatic processing of orders during system outages. It relies on an additional AWS Systems Manager Run Command step, which adds complexity.

Overall Explanation:

The best solution is to use an Auto Scaling group for the EC2 instances and configure the order system to send messages to an Amazon SQS queue. The EC2 instances will then consume messages from the queue. This decouples the applications and allows for reprocessing messages if failures occur due to networking issues or overloaded systems. This ensures that orders are processed even during outages, and no orders are lost.

[View Discussions](#)[Ask AI](#)

Question 63

Not Attempted

Database



A company runs an application on a large fleet of Amazon EC2 instances. The application reads and writes entries into an Amazon DynamoDB table. The size of the DynamoDB table continuously grows, but the application needs only data from the last 30 days. The company needs a solution that minimizes cost and development effort. Which solution meets these requirements?

- ☐ Use an AWS CloudFormation template to deploy the complete solution. Redeploy the CloudFormation stack every 30 days, and delete the original stack.
Redeploying CloudFormation stacks introduces unnecessary complexity and operational overhead.
- ☐ Use an EC2 instance that runs a monitoring application from AWS Marketplace. Configure the monitoring application to use Amazon DynamoDB Streams to store the timestamp when a new item is created in the table. Use a script that runs on the EC2 instance to delete items that have a timestamp that is older than 30 days.
Using an EC2 instance for monitoring and deletion adds infrastructure and maintenance efforts.
- ☐ Configure Amazon DynamoDB Streams to invoke an AWS Lambda function when a new item is created in the table. Configure the Lambda function to delete items in the table that are older than 30 days.
Configuring DynamoDB Streams with a Lambda function adds complexity and development/operational effort compared to using TTL.
- ☒ Extend the application to add an attribute that has a value of the current timestamp plus 30 days to each new item that is created in the table. Configure DynamoDB to use the attribute as the TTL attribute.
Using DynamoDB TTL allows you to define a per-item timestamp to determine when an item is no longer needed. Shortly after the date and time of the specified timestamp, DynamoDB deletes the

item from your table without consuming any write throughput. This solution requires minimal changes and no extra cost.

Overall Explanation:

DynamoDB Time to Live (TTL) allows you to define a per-item timestamp to determine when an item is no longer needed. Shortly after the date and time of the specified timestamp, DynamoDB deletes the item from your table without consuming any write throughput. This minimizes cost and development effort compared to other options involving CloudFormation, EC2 instances, or Lambda functions.

[View Discussions](#)[Ask AI](#)

Question 64

Not Attempted

Database



A company has a Microsoft .NET application that runs on an on-premises Windows Server. The application stores data by using an Oracle Database Standard Edition server. The company is planning a migration to AWS and wants to minimize development changes while moving the application. The AWS application environment should be highly available. Which combination of actions should the company take to meet these requirements? (Choose two.)

- ☐ Refactor the application as serverless with AWS Lambda functions running .NET Core.
Refactoring to serverless requires significant code changes and is not ideal when minimizing development efforts.
- ☒ Rehost the application in AWS Elastic Beanstalk with the .NET platform in a Multi-AZ deployment.
Rehosting in Elastic Beanstalk minimizes code changes and a Multi-AZ deployment ensures high availability.
- ☐ Replatform the application to run on Amazon EC2 with the Amazon Linux Amazon Machine Image (AMI).
Replatforming requires more overhead compared to rehosting and doesn't directly address minimizing development changes.
- ☐ Use AWS Database Migration Service (AWS DMS) to migrate from the Oracle database to Amazon DynamoDB in a Multi-AZ deployment.

Migrating to DynamoDB would require significant application changes due to the different data model (NoSQL vs. Relational).

- ☒ Use AWS Database Migration Service (AWS DMS) to migrate from the Oracle database to Oracle on Amazon RDS in a Multi-AZ deployment.

Migrating to Oracle on RDS with Multi-AZ maintains database compatibility and provides high availability without requiring application code changes.

Overall Explanation:

To minimize development changes and ensure high availability, the company should rehost the .NET application in Elastic Beanstalk in a Multi-AZ deployment and migrate the Oracle database to Oracle on Amazon RDS in a Multi-AZ deployment. This approach allows the application to run with minimal code modifications while leveraging AWS managed services for scalability and high availability. Migrating to DynamoDB or refactoring to Lambda would require significant code changes.

 View Discussions

 Ask AI

Question 65

Not Attempted

Database



A company currently runs a containerized application on a Kubernetes cluster in its on-premises data center, utilizing a MongoDB database for persistent data storage. To improve scalability and reduce operational burden, the company plans to migrate portions of this environment to AWS. Due to existing constraints, changes to the application code or existing deployment procedures are not permitted. The primary goal is to implement a solution on AWS that minimizes ongoing operational overhead. Which of the following options fulfills these requirements?

- ☐ Use Amazon Elastic Container Service (Amazon ECS) with Amazon EC2 worker nodes for compute and MongoDB on EC2 for data storage.

This solution increases operational overhead because it requires manual management and scaling of EC2 instances.

- ⊗ Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate for compute and Amazon DynamoDB for data storage.

This option necessitates significant code changes due to DynamoDB's different data model compared to MongoDB.

- ⊗ Use Amazon Elastic Kubernetes Service (Amazon EKS) with Amazon EC2 worker nodes for compute and Amazon DynamoDB for data storage.

Similar to Option B, this requires code changes for DynamoDB and managing EC2 worker nodes contributes to higher operational overhead.

- ✔ Use Amazon Elastic Kubernetes Service (Amazon EKS) with AWS Fargate for compute and Amazon DocumentDB (with MongoDB compatibility) for data storage.

This solution allows running the Kubernetes environment on AWS without changes using EKS. Fargate removes the need to manage EC2 instances, and DocumentDB provides MongoDB compatibility, minimizing changes and overhead.

Overall Explanation:

The best solution is to use Amazon EKS with Fargate for compute and Amazon DocumentDB for data storage. EKS supports running existing Kubernetes deployments on AWS without code changes. Fargate eliminates the need to manage EC2 instances, reducing operational overhead. DocumentDB provides MongoDB compatibility, removing the need for code changes at the data layer. Other options either require code changes (using DynamoDB) or introduce operational overhead (managing EC2 instances).

[View Discussions](#)[Ask AI](#)[← Back to Exam](#)

Marketing

Company

[Affiliate Program](#)

[Udemy Alternative](#)

[Unlimited Access](#)

[Whizlabs Alternative](#)

[Business Unlimited Access](#)

[TutorialsDojo Alternative](#)

[Support](#)

[Legal](#)

[Contact Us](#)

[Terms of service](#)

team@itcertifications.io

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITcertifications Technologies. All rights reserved.